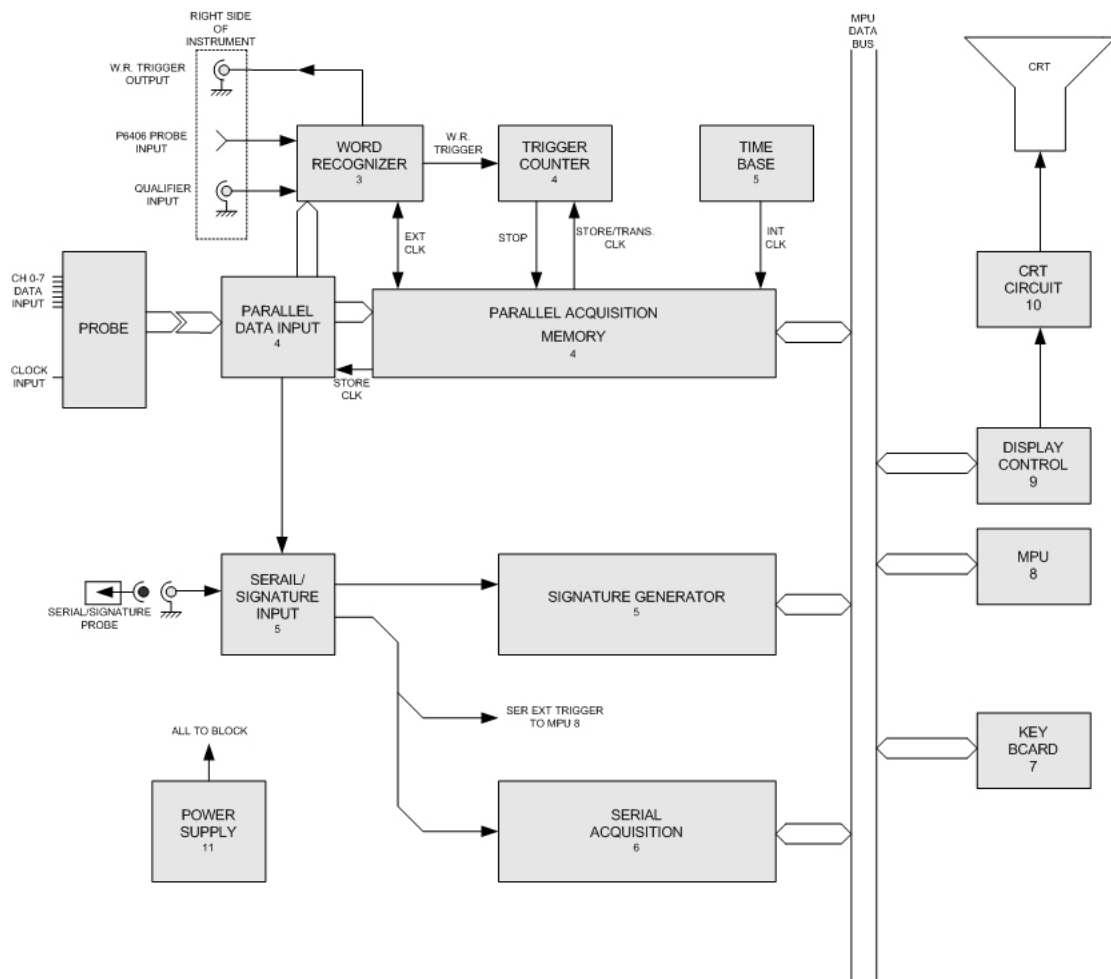


บทที่ 2

ทฤษฎีและหลักการ

2.1 โครงสร้างและการทำงานทั่วไปของลอจิกแอนะไลเซอร์

โครงสร้างและหลักการทำงานทั่วไปของเครื่องวิเคราะห์สัญญาณทางตรรกะสามารถแสดงดังบล็อกไดอะแกรมในรูปที่ 2-1 ซึ่งประกอบด้วยส่วนย่อยๆ ดังนี้



รูปที่ 2-1 โครงสร้างและการทำงานทั่วไปของเครื่องวิเคราะห์สัญญาณทางตรรกะ

2.1.1 ส่วนของสัญญาณด้านขาเข้าแบบขนาน (Parallel data input)

วงจรจะประกอบไปด้วย วงจรศักดาไฟฟ้าขีดเริ่มเปลี่ยน (Threshold voltage) , วงจรแปลงจาก ECT-TTL , วงจรหน่วงสัญญาณ , อินเวอร์เตอร์ , วงจรสุ่มสัญญาณตัวอย่าง (Sample/Latch) ซึ่งจุดประสงค์ ของส่วนนี้เป็นตัวปรับระดับของสัญญาณขาเข้าให้มีระดับที่เหมาะสมเพื่อนำไปประมวลผลต่อไป

2.1.2 ส่วนรู้จำของคำ (Word recognizer)

วงจรของส่วนรู้จำของคำจะให้สัญญาณขาออกเป็นลอจิก “1” (High) เมื่อสัญญาณทางตรรกะของสัญญาณรับเข้ามาจากแท่งรับสัญญาณ (Probe) เข้ากันได้กับสัญญาณขาที่เลือกไว้ (Qualifier input) ซึ่งสัญญาณจากส่วนรู้จำของคำนี้เป็นสัญญาณกระตุ้นแบบอะซิงโครนัส (Asynchronous trigger pulse)

2.1.3 ส่วนหน่วยความจำรับสัญญาณแบบขนาน (Parallel acquisition memory)

เป็นวงจรที่ประกอบไปด้วยหน่วยความจำ (RAM) , ตัวนับแอดเดรส (Address counter) ซึ่งหน่วยความจำนี้เป็นตัวเก็บสัญญาณขาเข้าซึ่งผ่านการปรับระดับของสัญญาณแล้ว และตัวนับแอดเดรสเป็นตัวเลือกบริเวณของแอดเดรสที่ต้องการพิจารณา

2.1.4 ส่วนหน่วงสัญญาณกระตุ้น (Trigger delay)

วงจรหน่วงสัญญาณกระตุ้นจะเป็นวงจรประกอบด้วย วงจรกระตุ้น (Trigger) และหน่วงสัญญาณต่างๆ เช่น Trigger delay counter , Trigger gate stage , Delay gate stage และอื่นๆ ซึ่งจะทำให้เกิดสัญญาณขาออกที่หน่วงเวลาการกระตุ้น (Delayed trigger output) เป็นสัญญาณที่ไปกระตุ้น วงจร Parallel Acquisition memory

2.1.5 วงจรคาบเวลา (Time base)

เป็นวงจรประกอบไปด้วยความถี่มาตรฐาน , วงจรแบ่งความถี่ ซึ่งหน้าที่ของวงจรจะเป็นส่วนให้สัญญาณนาฬิกาให้กับตัวเอ็มพียู (MPU) และวงจรควบคุมการแสดงผล

2.1.6 วงจรสัญญาณขาเข้าแบบอนุกรม (Serial / Signature input)

เป็นวงจรทำหน้าที่เปรียบเทียบสัญญาณขาเข้า ปรับแต่งระดับของสัญญาณให้เหมาะสม เพื่อเข้าสู่วงจรในส่วนอนุกรม (Serial) อื่นๆต่อไป

2.1.7 ตัวกำเนิดสัญญาณ (Signature generator)

จะเป็นตัวรับสัญญาณแบบอนุกรมเข้ามาถอดรหัส (Decode) ของสัญญาณขาเข้า และทำให้เป็นสัญญาณขาออกเป็น 16 บิต (16-bit output) และส่งเข้าไปยังแรม (RAM) แสดงผลในหน่วยเอ็มพียู (MPU) ต่อไป

2.1.8 วงจรรับสัญญาณขาเข้าแบบอนุกรม (Serial data acquisition)

วงจรรับสัญญาณขาเข้าแบบอนุกรม ประกอบด้วย Baud rate generator, USART (Programmable communication interface) , Data buffer ซึ่งวงจรนี้จะเป็นกระจายสัญญาณนาฬิกาแบบภายนอกและภายใน (Internal / External) และรับสัญญาณขาเข้าแบบอนุกรมแล้วแปลงเป็นสัญญาณขาออกแบบขนาน โดยส่งผ่าน Data buffer เข้าสู่ Data bus MPU ต่อไป

2.1.9 คีย์บอร์ด และเอ็มพียู (Keyboard and MPU)

คีย์บอร์ดจะเป็นตัวส่งสัญญาณ ไปยัง MPU โดยผ่าน Latch และ Gate เพื่อจัดเวลาให้เหมาะสม ส่วน MPU นั้นเป็นหน่วยของไมโครโปรเซสเซอร์ ซึ่งเป็นหน่วยประมวลผลต่าง ๆ นั้นเอง รวมถึง ROM, RAM , Decode

2.1.10 หน่วยควบคุมการแสดงผล (Display control)

จะเป็นวงจรประกอบขึ้นด้วย RAM ซึ่งจะเก็บข้อมูลที่แสดงผลออกมา ซึ่งได้รับการประมวลผลโดยเอ็มพียู (MPU) ไว้แล้ว เพื่อส่งออกไปทางจอภาพ

2.1.11 จอภาพ (CRT)

เป็นหน่วยประมวลผลเพื่อแสดงสัญญาณออกที่จอภาพ

2.1.12 วงจรจ่ายกำลัง (Power supply)

เป็นวงจรจ่ายกำลัง หรือ จ่ายศักดาไฟฟ้าแบบ DC

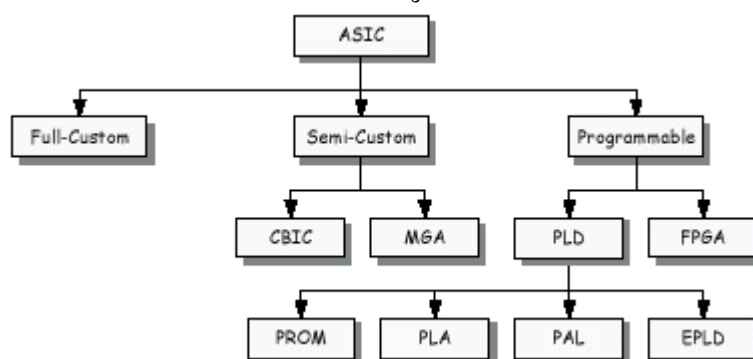
2.2 ความเป็นมาของ FPGA

ในช่วงก่อนทศวรรษ 1970 อุตสาหกรรมเซมิคอนดักเตอร์ได้ถูกปลูกให้ต้นตัวขึ้นหลังจากที่มีการประดิษฐ์วงจรรวมหรือไอซี ตัวแรกสำเร็จ ในยุคแรกนั้นไอซีขนาดเล็กหรือ SSI (Small-Scale Integration) ประกอบไปด้วยเกทดิจิทัลจำนวนไม่มากนัก (ประมาณ 1 ถึง 10 ตัว) ต่อมาได้มีการเพิ่มปริมาณของเกทดิจิทัลและฟังก์ชันทางลอจิกให้มากขึ้นจนเป็น MSI (Medium-Scale Integration) การพัฒนาไอซีเป็นไปอย่างต่อเนื่องจนมาถึงยุคของ LSI (Large-Scale Integration) ซึ่งเป็นยุคที่มีการสร้างไมโครโปรเซสเซอร์ตัวแรกขึ้น และในปัจจุบันเป็นยุคของ VLSI (Very Large-Scale Integration) ซึ่งเทคโนโลยีในการสร้างไอซีรุ่นหน้าจจะสามารถสร้างไมโครโปรเซสเซอร์ขนาด 64 บิต ที่มีหน่วยความจำเท่ากับหน่วยคำนวณทางคณิตศาสตร์ ของโฟลตติงพอยน์ (Floating-Point Arithmetic Units) รวมอยู่ภายในตัวมันและเนื่องจากการปรับปรุงเทคโนโลยีของกระบวนการสร้างชิปที่มีมาอย่างต่อเนื่องทำให้ขนาดของทรานซิสเตอร์ที่บรรจุอยู่ในไอซีมีขนาดเล็กลงเรื่อยๆ จนบางคนโดยเฉพาะ ในญี่ปุ่นใช้คำว่า ULSI (Ultra large Scale Integration) เพื่อใช้เรียกระดับของไอซีในปัจจุบันแต่คนส่วนมาก ยังมักนิยมเรียกเพียงแค่ VLSI

จากการปรากฏตัวของ VLSI ในช่วงทศวรรษ 1980 ทำให้วิศวกรเริ่มมีการออกแบบไอซีตามความต้องการของลูกค้าซึ่งใช้ใน ระบบที่เจาะจงนอกเหนือจากการใช้ไอซีมาตรฐานเพียงอย่างเดียว โดยไอซีเหล่านี้มีชื่อเรียกว่า ASIC : Application-Specific Integrated Circuit (ออกเสียงว่า เอ-ซิก) ซึ่งตัวอย่างของ ASIC ได้แก่ ชิพไอซีที่ใช้ สำหรับตุ๊กตาของเล่นพูดได้ ดาวเทียม และ ชิพที่อยู่ในบรรจุด้วยไมโครโปรเซสเซอร์กับอุปกรณ์ทางลอจิกอื่นๆ

2.2.1 ประเภทของ ASIC

ASIC แบ่งเป็น 3 ประเภทใหญ่ๆ คือ Full-custom, Semi-custom และ Programmable ดังรูปที่ 2-2



รูปที่ 2-2 ประเภทของ ASIC

2.2.1.1 Full-custom

ASIC ประเภทนี้ลูกค้าจะเป็นผู้ออกแบบเซลล์ลอจิก (เช่น แอนด์เกต ออร์เกต มัลติเพล็กซ์เลอร์ และ ฟลิปฟล็อป) และลักษณะ การจัดวางอุปกรณ์บนตัวไอซีรวมถึงหน้ากากสำหรับควบคุมการเชื่อมและสร้างชั้น สาร (Mask) ต่างๆ ที่ใช้ในการทำไอซีเอง ดังนั้นค่าใช้จ่ายในการออกแบบและการผลิตจะสูงมาก

2.2.1.2 Semi-custom

ASIC ประเภทนี้เซลล์ลอจิกจะถูกออกแบบเอาไว้ก่อนแล้วในรูปแบบของไลบรารีและลูกค้าจะเป็น ผู้ออกแบบ Mask ต่างๆเอง ตัวอย่างของไอซีประเภทนี้ได้แก่ Standard-Cell-Based ASIC และ Masked Gate-Array-Based ASIC

ก) Standard-Cell-Based ASIC

ไอซีประเภทนี้จะมีพื้นที่สำหรับจัดวางเซลล์ลอจิกมาตรฐานซึ่งถูกออกแบบเอาไว้แล้ว ในบางครั้ง เซลล์ มาตรฐานเหล่านี้จะถูกนำมาประกอบกันเป็นเซลล์ที่มีขนาดใหญ่ขึ้นเรียกว่า Megacell สำหรับการออกแบบนั้นผู้ออกแบบจะทำเพียงแค่กำหนดตำแหน่งของเซลล์มาตรฐานและการเชื่อมต่อภายในของแต่ละเซลล์ เท่านั้นแต่อย่างไรก็ดีเซลล์ต่างๆ เหล่านี้สามารถวางที่ตำแหน่งใดๆ ก็ได้บนแผ่นเวเฟอร์ซิลิกอน นั่นก็หมายความว่าชั้น Mask จะถูกจัดวางตามความต้องการของผู้ออกแบบ

ข) Masked Gate-Array-Based ASIC

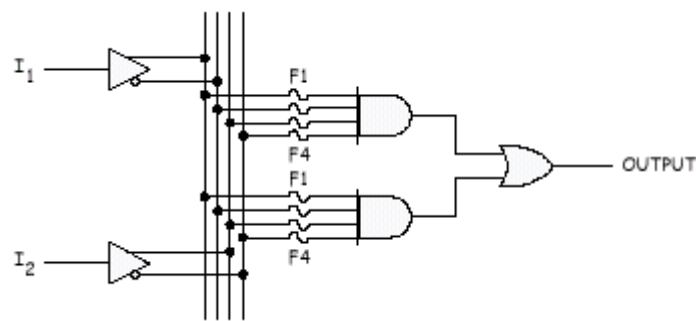
ไอซีชนิดนี้จะมีทรานซิสเตอร์หรือเกตถูกสร้างมาในลักษณะของอะเรย์สองมิติบนแผ่นเวเฟอร์ ซิลิกอน และผู้ออกแบบจะทำการออกแบบ Mask เพื่อใช้สำหรับกำหนดการต่อเชื่อมของทรานซิสเตอร์แต่ละ ตัว

2.2.1.3 Programmable

ASIC ประเภทนี้เซลล์ลอจิกจะถูกออกแบบไว้ก่อนเช่นเดียวกับ Semi-Custom แต่ชั้นของ Mask จะไม่สามารถเปลี่ยนแปลงได้ ตามความต้องการของผู้ออกแบบ ไอซีประเภทนี้ยังแบ่งออกเป็น 2 ชนิดคือ Programmable Logic Device (PLD) และ Field Programmable Gate Array (FPGA)

ก) Programmable Logic Device (PLD)

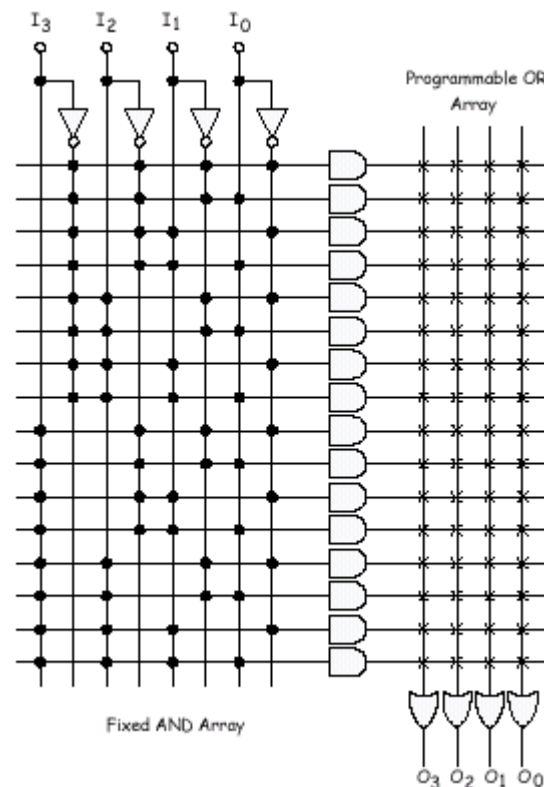
มีโครงสร้างภายในเป็นวงจรพื้นฐานทางด้านลอจิกต่อกันอยู่เป็นกลุ่มซึ่งมีทั้งวงจรคอมบิเนชัน (Combination) และซีควเอนเชียล (Sequential) สำหรับเทคโนโลยีของวงจรที่ใช้สร้าง PLD จะมีทั้ง TTL, ECL และ CMOS ตามความเหมาะสมของแต่ละระบบ ไอซี PLD ทุกชนิดมีหลักการพื้นฐานของวงจรภายในที่เหมือนกันโดยมี วงจรคอมบิเนชันที่เป็นผลคูณร่วม บวก (Sum of product)ประกอบไปด้วยชุดของแอนด์เกต ต่อร่วมกับออร์ เกตและในการ โปรแกรมจะเป็นการเลือกเอาอินพุตภายในของแอนด์เกตกับสัญญาณอินพุต ใดบ้างที่จะต้อง ต่อ ถึงกันซึ่งมีทั้งจากภายนอกและสัญญาณป้อนกลับจากเอาต์พุตภายในเอง เช่น การติดต่อ อินพุตของออร์ เกตกับเอาต์พุตของแอนด์เกตตัวต่างๆ สำหรับการโปรแกรมทางกายภาพนั้นอินพุต ต่างๆ ของ อุปกรณ์ทุกตัว จะถูกต่อผ่านฟิวส์เข้ากับแหล่งสัญญาณ ซึ่งถ้าไม่ต้องการใช้สัญญาณ ใดก็จะตัดฟิวส์ตัวนั้นทิ้งทำให้สามารถ โปรแกรมได้เพียงครั้งเดียว ไอซี PLD บางชนิดใช้ มอสทรานซิสเตอร์แทนฟิวส์ทำให้สามารถ โปรแกรมโดย ใช้กระแสไฟฟ้าและสามารถลบแล้ว โปรแกรมเข้าไปใหม่ได้อีก สำหรับไอซีในตระกูล PLD ได้แก่ PROM, PAL, PLA และ EPLD



รูปที่ 2-3 วงจรพื้นฐานของ PLD ซึ่งอยู่ในรูปผลคูณร่วมบวก

- PROM (Programmable Read Only Memory)

PROM คือหน่วยความจำประเภท ROM ซึ่งนับว่าเป็นไอซี PLD ชนิดหนึ่งซึ่งวงจรภายใน ของ PROM ประกอบไปด้วยอะเรย์ของแอนด์และออร์เกท (And - Or Array) ผลลัพธ์ที่ขา ด้าเอาต์พุทสามารถแสดงได้ใน สมการของฟังก์ชันผลคูณร่วมบวก (Sum of product) ของสัญญาณอินพุทที่ขาแอดเดรส



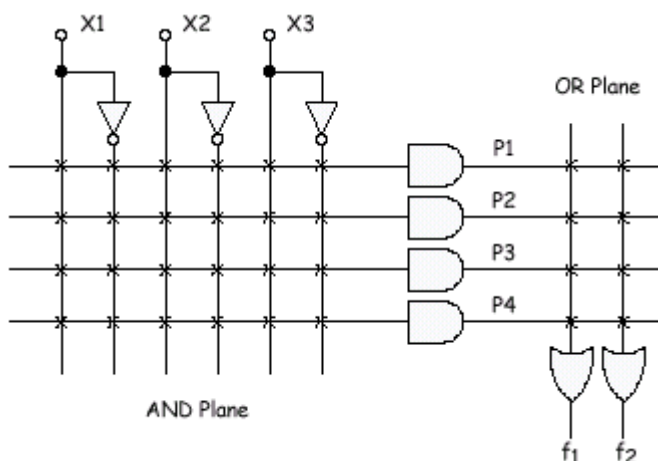
รูปที่ 2-4 ลักษณะของ PROM เมื่อเปรียบเทียบกับเป็นวงจรในรูปผลคูณร่วมบวก

รูปที่ 2-4 แสดงถึงลักษณะการเชื่อมต่อแอนด์เกทและออร์เกทของ PROM ขนาด 16x4 บิต วงจรทางด้านซ้ายบนสุดเป็นแอนด์เกทจะให้ผลคูณ (Product) ของกรณีที่อินพุทเป็น 0000 แอนด์เกทที่อยู่ถัดลงมาเป็นผลคูณของกรณีที่อินพุทเป็น 0001, 0010, ...จนถึงตัวล่างสุดคือ ผลคูณในกรณีที่อินพุทเป็น 1111 ซึ่งสำหรับ PROM ที่มีจำนวนอินพุท n ตัวจะมีค่าอินพุทที่เป็นไปได้ทั้งหมดเท่ากับ 2^n และค่าอินพุทเหล่านี้จะถูกจัดวางอยู่ในส่วนอะเรย์ของ AND ซึ่งไม่สามารถแก้ไขได้ แต่ในส่วนของ OR จะเป็นส่วนที่อนุญาตให้ทำการโปรแกรมได้ และเนื่องจากการที่ด้าน AND ของ PROM มีการคอมบินชันของอินพุทที่เป็นไปได้ทั้งหมด

ดังนั้นผู้ออกแบบจึงไม่จำเป็นต้องทำการลดรูปของฟังก์ชันลอจิกที่ออกแบบไว้เลยแต่อย่างไรก็ดีการกระทำเช่นนี้อาจทำให้เกิดจำนวนวงจรที่ไม่มีประสิทธิภาพจำนวนมากบนตัว ชิปได้

- PLA (Programmable Logic Array)

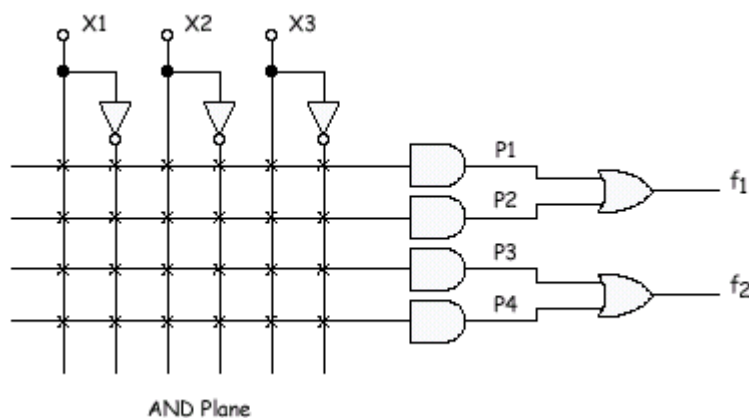
ลักษณะเด่นของ PLA คือสามารถโปรแกรมการเชื่อมต่อได้ทั้งทางด้าน AND และด้าน OR ทำให้มีความยืดหยุ่นในการใช้งานมาก แต่อย่างไรก็ดีข้อเสียที่เห็นได้อย่างชัดเจน ของ PLA คือความยุ่งยากในการสร้างและคุณสมบัติทางด้านความเร็วที่ลดลงเนื่องจาก สัญญาณจะต้องวิ่งผ่านอะเรย์ของ AND และ OR



รูปที่ 2-5 วงจรพื้นฐานภายในของ PLA

- PAL (Programmable Array Logic)

PAL มีลักษณะโครงสร้างที่ใกล้เคียงกับ PROM และ PLA มาก แต่การโปรแกรม PAL จะสามารถทำได้เพียงด้าน AND เท่านั้น



รูปที่ 2-6 วงจรพื้นฐานภายในของ PAL

- EPLD (Erasable Programmable Logic Device)

EPLD เป็นอุปกรณ์ที่สามารถทำการโปรแกรมได้หลายครั้งซึ่งเหมาะสำหรับการ ทำวงจรต้นแบบ สำหรับเทคโนโลยีที่ใช้ในการสร้างจะเหมือนกับ CMOS EPROM คือ ใช้มอสทรานซิสเตอร์เชื่อมต่อระหว่าง สัญญาณอินพุตกับจุดที่ต้องการแทนการ ใช้ฟิวส์แบบเดิมทำให้สามารถโปรแกรมการต่อวงจรภายในอุปกรณ์ ด้วยการจ่าย ไฟฟ้าตามขนาดที่กำหนดไว้และลบได้โดยใช้แสงอัลตราไวโอเลตฉาย ผ่านช่อง หน้าต่างกระจก ของตัวชิป

ข) Field-Programmable Gate Array (FPGA)

เป็นอุปกรณ์ที่มีความซับซ้อนมากกว่า PLD ไปอีกระดับหนึ่ง ซึ่งในความเป็นจริงแล้ว PLD และ FPGA แตกต่างกันอย่างสิ้นเชิง สำหรับ FPGA แล้วนับว่าเป็นอุปกรณ์ตัวใหม่ในตระกูลของ ASIC ซึ่งมีการเจริญเติบโตอย่างรวดเร็วและมีบทบาทที่สำคัญในการเข้ามาแทนที่ระบบอิเล็กทรอนิกส์ที่ใช้ TTL โครงสร้างภายในของ FPGA ประกอบไปด้วยอะเรย์ของลอจิกเกทต่างๆมากมาย ซึ่งในปัจจุบันความจุเกทภายใน ตัวชิพ FPGA ได้เพิ่มขึ้น จากระดับไม่กี่พันตัวจนถึงระดับล้านตัวซึ่งสามารถรองรับวงจรดิจิทัลที่มีความสลับซับซ้อนได้เป็นอย่างดี นอกจากนี้ในด้านการออกแบบพัฒนาและทดสอบก็ทำได้ง่ายซึ่งในปัจจุบัน การออกแบบวงจรโดยใช้ FPGA กำลังเป็นที่นิยมและมีแนวโน้มที่จะนำมาใช้งานมากขึ้นเรื่อย

2.3 สถาปัตยกรรม FPGA

ในปัจจุบันมี FPGA อยู่ 4 ชนิดที่วางขายอยู่ในท้องตลาดได้แก่ Symmetrical Array, Row-Based, Hierarchical PLD และ Sea-of-Gates ซึ่งแต่ละชนิดก็มีลักษณะการเชื่อมต่อภายในและการโปรแกรม ที่แตกต่างกันไป นอกจากนี้ในการแบ่งประเภทของ FPGA อาจแบ่งได้ตามเทคโนโลยีที่ใช้ในการโปรแกรม ซึ่งมีอยู่ 2 แบบคือ การโปรแกรมโดยการทำให้เกิดการเปลี่ยนแปลงทางกายภาพของตัวชิพ และการโปรแกรม โดยการใช้หน่วยความจำ



รูปที่ 2- 7 แสดง FPGA แต่ละประเภท

2.3.1 การโปรแกรมโดยการทำให้เกิดการเปลี่ยนแปลงทางกายภาพ

- **Fuse** เป็นวิธีการโปรแกรมที่สามารถทำได้เพียงครั้งเดียว ซึ่งหลังจากที่โปรแกรมแล้ว จุดเชื่อม ต่อจะขาดจากกัน
- **Anti Fuse** เป็นวิธีการโปรแกรมที่คล้ายกับแบบ Fuse แต่ต่างกันที่หลังจากทำการโปรแกรม แล้วจุดเชื่อมต่อจะเชื่อมถึงกัน

2.3.2 การโปรแกรมโดยใช้หน่วยความจำ

- **EEPROM Based FPGA**

FPGA ที่ใช้การโปรแกรมแบบนี้มักเรียกว่า CPLD ซึ่งเทคโนโลยีที่ใช้จะเหมือนกับ EEPROM ทำให้มีความจุของเกทต่ำ โดยทั่วไปจะน้อยกว่า 20,000 เกท แต่ข้อดีของ

EEPROM Based FPGA คือสามารถเก็บข้อมูลที่โปรแกรมลงไปได้โดยไม่ต้องมีไฟเลี้ยง และในการโปรแกรมจะใช้ทรานซิสเตอร์ 1 ตัวต่อ 1 บิต ซึ่งการโปรแกรมสามารถทำได้ประมาณ 10,000 ครั้ง

- **SRAM Based FPGA**

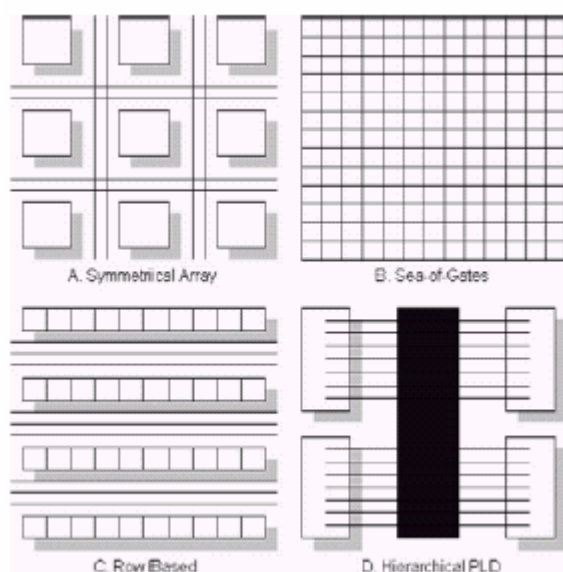
FPGA แบบนี้จะใช้เทคโนโลยีในการโปรแกรมเหมือนกับ SRAM (Static RAM) ทำให้สามารถ โปรแกรมซ้ำได้โดยไม่ต้องจำกัดจำนวนครั้ง นอกจากนี้ยังมีความจุของเกทในระดับปานกลางถึงสูงมาก (ประมาณ 10,000 - 1,000,000 เกท) ซึ่งข้อดีของ SRAM Based FPGA คือใช้เวลาในการ โปรแกรมน้อย (ระดับ nsec) การ โปรแกรมทำได้ง่ายเทียบได้กับการเขียน SRAM ทั่วไป และ เหมาะสำหรับการออกแบบวงจรที่มีความสลับซับซ้อน ส่วนข้อเสียคือไม่สามารถเก็บโปรแกรมใน ภาวะที่ไม่มีไฟเลี้ยงได้ ดังนั้น FPGA ชนิดนี้จึงมักใช้ควบคู่กับ ROM เพื่อเก็บโปรแกรมและทำการ โหลดโปรแกรมลงในตัวชิปในขณะที่เริ่มต้นใช้งาน

2.4 โครงสร้างภายในของ FPGA

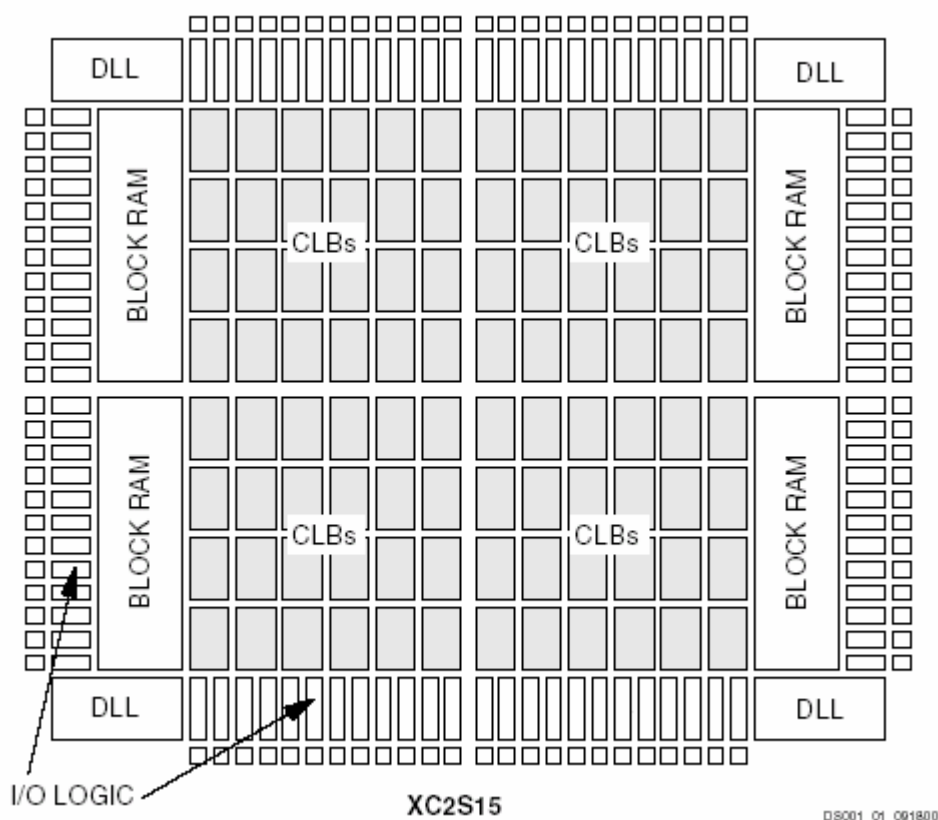
ลักษณะโครงสร้างภายในของ FPGA จะเป็นอะเรย์ของบล็อกลอจิกที่สามารถทำการ โปรแกรมได้

FPGA ย่อมาจากคำว่า “Filed Programmable Gate Array” ซึ่งในที่นี้จะกล่าวถึงสถาปัตยกรรมของ FPGA โครงสร้างและหลักการทำงานของ FPGA บริษัท Xilinx ตระกูล XC2S50 ซึ่งใช้สถาปัตยกรรมของ FPGA แบบ Symmetrical Array ดังนี้

2.4.1 สถาปัตยกรรม FPGA แบ่งออกเป็น 4 ประเภท ด้วยกัน คือ Symmetrical Array, Row Based, Hierarchical PLD และ Sea of Gate ดังรูปที่ 2-8 ซึ่ง FPGA ของ Xilinx ตระกูล XC2S50 จะใช้สถาปัตยกรรมที่เรียกว่า Symmetrical Array มี “Static Random Access Memory” (SRAM) เป็นส่วนเก็บข้อมูลที่โปรแกรมรวมอยู่ภายใน Chip และสามารถเพิ่ม PROM เป็นส่วนที่เก็บข้อมูลภายนอกเพื่อให้ FPGA สามารถ Reconfigured ตัวเองได้ภายใน chip ประกอบด้วยบล็อกของลอจิก local routing เรียกว่า “Xilinx Configurable Logic Block”(CLB) โดย แต่ละบล็อกจะต่อถึงกันโดยผ่าน Switch Matrix แสดงดังรูปที่ 2-9



รูปที่ 2-8 แสดงสถาปัตยกรรม FPGA

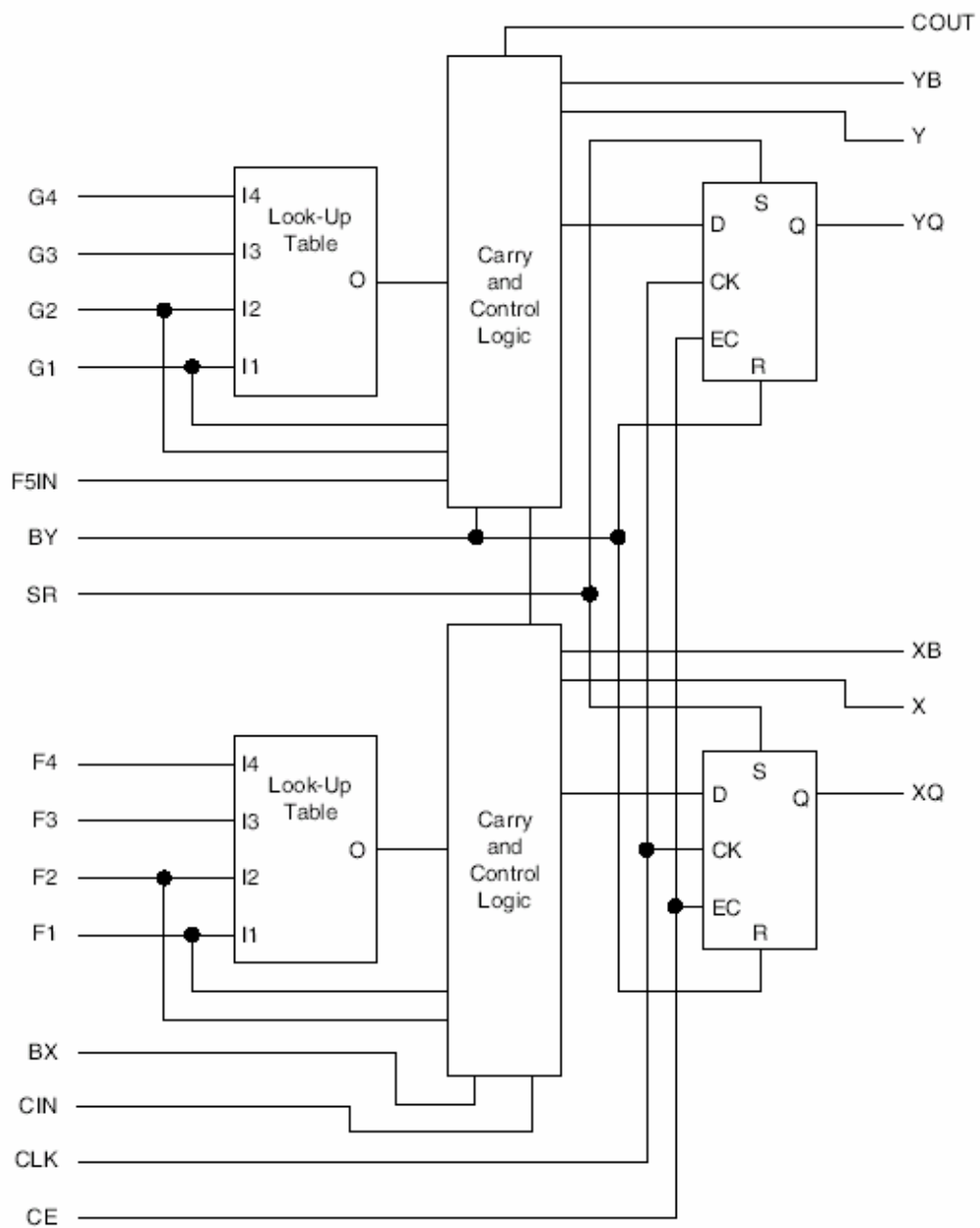


รูปที่ 2-9 แสดง Symmetrical Array FPGA ของ Xilinx ตระกูล XC2S50

โครงสร้างและหลักการทำงานของ FPGA ตระกูล XC2S50 สำหรับโครงสร้างของ FPGA แสดง ดังรูปที่ 2.9 ภายใน Logic Block ประกอบด้วย อุปกรณ์ที่สามารถโปรแกรมได้เรียกว่า “Configurable Logic

Block”(CLB) มีบล็อกรับอินพุตกับเอาต์พุต Array เรียงกันไปตามของ Array ดังรูปที่ 2-9 เรียกว่า “Input / Output Block” (IOB) ซึ่งการต่อภายใน Array ของ CLB และ IOB แต่ละตัวผ่านทางช่องทางเดินไฟ (Wire) โดยมีอุปกรณ์ที่สามารถโปรแกรมไว้วางขึ้นอยู่ในแต่ละช่วงดังรูปที่ 2-9 เรียกอุปกรณ์นี้ว่า Switch Matrix

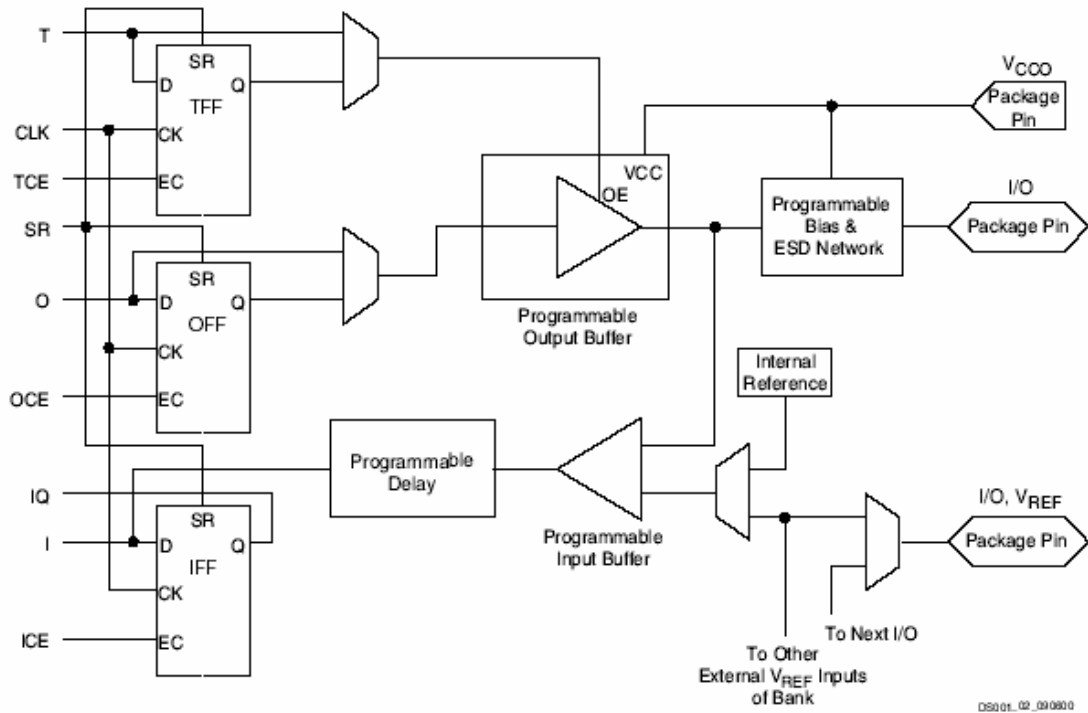
2.4.2 วงจรอย่างง่ายของ CLB ดังรูปที่ 2-10 ใน 1 บล็อกสามารถรับอินพุตได้ 13 สัญญาณ ซึ่งรวมถึง สัญญาณนาฬิกา (K), H1, DIN, Global Set/Reset , EC และมีเอาต์พุต 4 สัญญาณ คือ X, Y, XQ, YQ จากรูปมี lookup table 2 ตัว คือ lookup table for G' และ lookup table for F' ในการรวมสัญญาณ G' F' และ H1 เอาต์พุตของ lookup table ทั้ง 3 จะส่งให้ Multiplexer เป็นตัวเลือกสัญญาณ โดยที่สัญญาณเอาต์พุต x จะมี Multiplexer เลือกสัญญาณระหว่าง F' กับ H1 สัญญาณเอาต์พุต Y จะเลือกสัญญาณระหว่าง G' กับ H' ในส่วนของสัญญาณเอาต์พุต XQ และ YQ มีส่วนควบคุมที่เหมือนกัน คือ จะมี Multiplexer ตัวที่ 1 ในการเลือกสัญญาณระหว่าง DIN, G' ,F' และ H' ส่งสัญญาณที่เลือกแล้วมาเข้าอินพุตของ D Flip-Flop ที่ถูกควบคุม โดยตัวควบคุมการ Set/Reset จากสัญญาณภายนอก ซึ่ง D Flip-Flop จะถูกควบคุมอีกครั้งโดยการเลือกสัญญาณของ Multiplexer ตัวที่ 2 ที่เลือกสัญญาณระหว่าง EC ภายนอกกับลอจิก “1”



DS001_04_091400

รูปที่ 2-10 แสดงวงจรของ Configurable Logic Block (CLB)

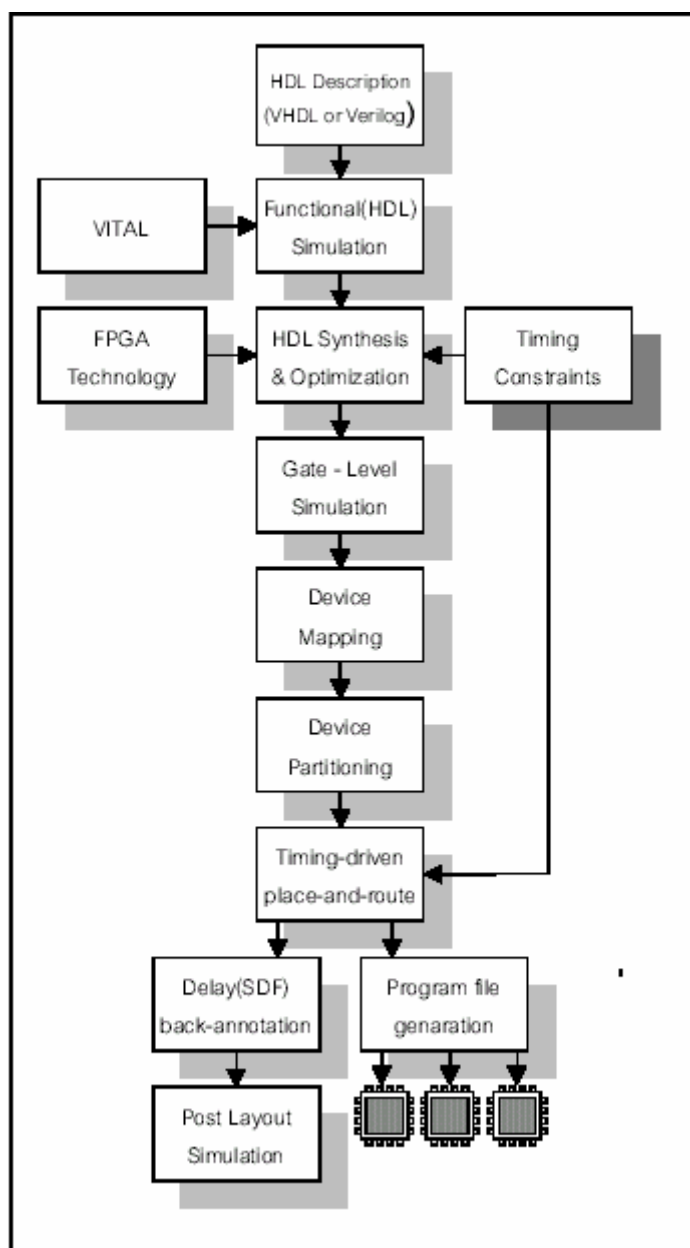
2.4.3 Input / Output Block (IOB) สามารถโปรแกรมการทำงานได้ จากรูปที่ 2-11 แสดงโครงสร้างอย่างง่าย ซึ่งในส่วนเอาต์พุตของ IOB จะส่งเอาต์พุต O ที่เก็บอยู่ใน D Flip-Flop ให้กับขา I/O โดยที่ Tri-State(TS) เป็นตัวควบคุมขา I/O ที่จะให้อินพุตหรือเอาต์พุต



รูปที่ 2-11 แสดงโครงสร้างของ Input / Output Block (IOB)

ในส่วนอินพุตของ IOB สัญญาณที่ขา I/O จะผ่านเข้ามาทาง Input buffer ผ่าน D Flip-Flop ส่งไปยัง Input data 1 และ Input data 2 ของตัว FPGA ต่อไป

2.5 แนวทางการออกแบบวงจรด้วย FPGA (FPGA Design)



รูปที่ 2.12 แสดงขั้นตอนการออกแบบโดยใช้ FPGA

ออกแบบโดยใช้ภาษาอธิบายฮาร์ดแวร์ (Hardware Description Language)

ในการออกแบบวงจรดิจิทัล นั้น ทำได้โดยการวาดวงจร (Schematic drawing) หรือใช้ภาษาอธิบายฮาร์ดแวร์ (Hardware Description Language) ในขั้นตอนนี้เป็นขั้นตอนที่ไม่แตกต่างกันระหว่างการออกแบบด้วย FPGA และ ASIC ในกรณีที่ใช้ภาษาอธิบายฮาร์ดแวร์ แต่ในกรณีที่ออกแบบโดยวิธีการวาดวงจรจะต่างกันโดยที่การทำวิธีนี้ผู้ทำการออกแบบต้องคำนึงถึงเทคโนโลยีที่จะใช้ซึ่งแต่ละเทคโนโลยีก็มีความแตกต่างกันไป จะเห็นได้ว่าออกแบบโดยใช้ภาษาอธิบายฮาร์ดแวร์ทำได้สะดวกกว่า เพราะการทำวิธีนี้ผู้ออกแบบไม่ต้องมาคำนึงถึงเทคโนโลยีที่จะใช้ และที่สำคัญการออกแบบโดยวิธีนี้สามารถแก้ไข (Model)

หรือเปลี่ยนเทคโนโลยีได้สะดวกกว่าเพราะไม่ต้องวาดวงจรใหม่ นั่นคือการออกแบบโดยใช้ภาษาอธิบายฮาร์ดแวร์ทำให้โมเดลที่ได้ไม่ขึ้นกับเทคโนโลยี สำหรับภาษาอธิบายฮาร์ดแวร์ที่ใช้ก็มี VHDL หรือ Verilog ในการเขียน (code) นั่นสิ่งที่จะต้องคำนึงถึงก็จะต้องเขียนอย่างไรจึงจะสามารถสังเคราะห์เป็นวงจรได้และให้โมเดลที่มีคุณสมบัติถูกต้องตามข้อกำหนด เพราะลักษณะการเขียนโค้ดจะมีผลโดยตรงกับโมเดลที่ได้ เนื่องจากการสังเคราะห์วงจรนั้น ซอฟต์แวร์สังเคราะห์วงจร (Synthesis Tools) จะสังเคราะห์วงจรตามโค้ด เช่นในการเขียนโค้ด VHDL อธิบายการทำงานของโมเดลวงจรอันเดียวกัน แต่เขียนโค้ดในลักษณะที่ต่างกัน เมื่อสังเคราะห์จะได้วงจรต่างกัน และจากวงจรที่ต่างกัน เมื่อนำไปทำต้นแบบด้วย FPGA หรือการทำ ASIC แล้วจะได้ไอซี ที่มีคุณสมบัติต่างกันทั้งในด้านของทางขนาดหรือความเร็ว (area and time) ดังนั้นการออกแบบในขั้นตอนนี้ออกแบบต้องระมัดระวังเป็นพิเศษ ส่วนการที่จะเขียนโค้ดในลักษณะใด เพื่อให้ได้ผลลัพธ์ที่ดีที่สุดนั้นก็ขึ้นอยู่กับประสบการณ์ของผู้ออกแบบ

2.5.1 การสังเคราะห์วงจร (Logic Synthesis)

ในขั้นตอนนี้จะใช้ซอฟต์แวร์สังเคราะห์วงจร (Synthesis tools) ทำการสังเคราะห์โค้ด VHDL หรือ Verilog เพื่อให้ได้เป็นวงจรขึ้นมา ซึ่งทำได้โดยใช้ซอฟต์แวร์อีกเช่นกันแต่ต้องตรวจสอบด้วยว่า ซอฟต์แวร์ นั้น ๆ สนับสนุนเทคโนโลยี FPGA (FPGA Library) ที่ต้องการใช้หรือไม่ ตัวอย่าง FPGA ที่มีการใช้งานแพร่หลาย เช่น FPGA ของบริษัท Xilinx และบริษัท Altera ในการทำ FPGA นั้นมีซอฟต์แวร์หลายตัวที่สามารถใช้ได้ เช่น Synopsys, Exemplar ฯลฯ ในขั้นตอนนี้ซอฟต์แวร์วงจรจะแปลงโค้ด VHDL และออปติไมซ์ (Optimization) เพื่อให้ได้วงจรตามเทคโนโลยีที่เลือกใช้ ในการสังเคราะห์วงจรเพื่อทำ FPGA นั้นวงจรระดับเกต (gate-level) ไม่เหมาะสมกับโครงสร้างที่มีอยู่ในอุปกรณ์ FPGA ดังนั้นในการออปติไมซ์นั้นซอฟต์แวร์สังเคราะห์วงจร จะต้องทำการออปติไมซ์ให้ได้เป็นวงจรที่ประกอบด้วยกลุ่มของลอจิก (logic) ที่เหมาะสมกับอุปกรณ์ FPGA นั้นๆ จึงจะทำให้ผลลัพธ์ที่ได้มีประสิทธิภาพ และในขั้นตอนการสังเคราะห์วงจรนี้ผู้ออกแบบสามารถกำหนดข้อบังคับสำหรับโมเดลได้เช่น ข้อบังคับในเรื่องของเวลา (timing constraints) หรือข้อบังคับในเรื่องของพื้นที่ (area) หรือกำหนดชนิดและตำแหน่งของ I/O ซึ่งข้อบังคับเหล่านี้จะถูกนำไปใช้ในขั้นตอนออปติไมซ์เพื่อให้วงจรที่ได้เป็นไปตามที่กำหนด ส่วนสำคัญในการออปติไมซ์คือการเทียบ (mapping) โมเดลให้เข้ากับเทคโนโลยีที่ใช้เพื่อให้วงจรที่เหมาะสมกับโครงสร้างสถาปัตยกรรมภายในอุปกรณ์ FPGA ในกรณีของ Xilinx FPGA จะเทียบโดยใช้วิธี LUT(Look Up Table)

เมื่อทำการสังเคราะห์เสร็จแล้วซอฟต์แวร์สังเคราะห์วงจรจะมีการรายงานผลว่าโมเดลที่ออกแบบไปเป็นอย่างไร เช่น มีความหน่วง(delay) เท่าไร ใช้ทรัพยากรต่างFPGA อะไรบ้าง เมื่อมาถึงขั้นตอนนี้ผู้ออกแบบก็จะทราบว่าโมเดลเป็นไรใหม่จนกว่าจะเป็นไปตามที่กำหนด ในขั้นตอนการสังเคราะห์นี้มีเทคนิคต่างๆ ที่ผู้ออกแบบนำมาใช้เพื่อให้โมเดลเป็นไปตามข้อบังคับที่กำหนด

2.5.2 การแบ่งวงจร (Partitioning)

ขั้นตอนนี้เป็นการแบ่งวงจรที่ได้จากการสังเคราะห์ เป็นส่วนย่อยๆ สำหรับลงใน CLBs, IBOS หรือองค์ประกอบอื่นๆ ภายในอุปกรณ์ FPGA สำหรับเกณฑ์ที่ใช้ในการแบ่งคือให้แต่ละส่วนที่จะแยกออกจากกันมีจำนวนสัญญาณที่เชื่อมต่อกันระหว่างกันน้อยที่สุดเท่าที่จะทำได้ เพื่อช่วยลดความหนาแน่นในตอนทำการเชื่อมต่อสัญญาณ (routing) ในขั้นตอนนี้จะใช้ซอฟต์แวร์ทำ โดยซอฟต์แวร์จะเทียบส่วนประกอบของวงจรเช่น

เกท (gate), ฟลิปฟล็อป (flip-flop) ลงในทรัพยากรต่างๆ ที่มีอยู่ภายในอุปกรณ์ FPGA (CLBs, IOBs, BUFT และ edge decoder)

หลังจากทำขั้นตอนนี้เสร็จแล้วผู้ออกแบบสามารถที่จะทราบว่าวงจรใช้จำนวนทรัพยากรภายในอุปกรณ์ FPGA ไปเท่าไร ส่วนข้อมูลทางเวลานั้นผู้ออกแบบจะทราบเฉพาะความหน่วงภายในแต่ละส่วนเท่านั้น หรือที่เรียกว่าความหน่วงลอจิก (logic delay) ส่วนซอฟต์แวร์ที่ใช้ในซอฟต์แวร์ที่ใช้ในขั้นตอนนี้นั้นเช่น M1 ของ Xilinx ซึ่งซอฟต์แวร์ตัวนี้จะรวมเอาซอฟต์แวร์ย่อยอื่นๆ อีกเพื่อให้การทำ PPR (Partitioning , Placement & Routing) เป็นไปอย่างต่อเนื่อง

2.5.3 การวางอุปกรณ์(Placement)

ขั้นตอนนี้คือการเลือกที่ตั้งของแต่ละส่วนของวงจรที่ผ่านการแบ่งวงจร (partitioning) มาแล้วว่าควร จะอยู่ ณ ตำแหน่งไหนในอุปกรณ์ FPGA เพื่อให้ได้ผลลัพธ์ที่ดีที่สุด เช่น วงจรส่วนไหนควรอยู่ใกล้กันเพื่อจะ ได้ค้นหาเส้นทาง (Route) ได้ง่ายหรือช่วยลดความหน่วง จะเห็นได้ว่าตำแหน่งภายในอุปกรณ์ FPGA นั้นมีความสำคัญเพราะถ้าจัดวางวงจรลงในตำแหน่งที่ไม่เหมาะสมแล้วจะทำให้ความหน่วงเพิ่มขึ้นหรือ Router ทำการค้นหาเส้นทางสัญญาณได้ไม่หมด

การวางอุปกรณ์ที่ดีควรวางส่วนต่างให้อยู่ใกล้กันโดยเฉพาะส่วนที่มีการเชื่อมต่อสัญญาณด้วยกัน นอกจากนั้นการกำหนดตำแหน่งขา I/O (I/O pin) ตามตำแหน่งขา I/O ของ FPGA บนแผ่น PCB ก็จะมีผลโดย ตารางเลขคือซอฟต์แวร์ จะวาง I/O ลงในตำแหน่งที่ผู้ออกแบบกำหนด ซึ่งบางครั้งตำแหน่งที่กำหนดไปไม่เหมาะสม ดังนั้นการกำหนดขา I/O ควรกำหนดตำแหน่งให้เหมาะสม หรือไม่ก็ให้ซอฟต์แวร์จัดการเอง ในกรณีที่ใช้ซอฟต์แวร์ M1 ผู้ออกแบบสามารถแก้ไขตำแหน่งใหม่ได้แต่ควรกระทำด้วยความระมัดระวังเป็นอย่างยิ่ง แต่วิธีที่ดีที่สุด สำหรับเกณฑ์ที่ใช้ในการตัดสินใจคือ ความหน่วงที่ได้หลังจากทำการค้นหาเส้นทางแล้ว หรือเรียกว่า routing delay

2.5.4 การเชื่อมต่อสัญญาณ (Routing)

ในขั้นตอนนี้เป็นเชื่อมต่อสัญญาณระหว่างองค์ประกอบต่างๆ ภายในอุปกรณ์ FPGA เช่นระหว่าง CLBs หรือระหว่าง CLBs กับ IOBs ขั้นตอนนี้จะทำการต่อเนื่องจากการวางอุปกรณ์ ในกรณีที่ทำการวางอุปกรณ์ไว้ไม่ดี ซอฟต์แวร์ก็จะทำการเชื่อมต่อสัญญาณได้ไม่หมด (เนื่องจากจำนวนทรัพยากร (resource) สำหรับเชื่อมต่อสัญญาณนั้นมีอยู่จำกัด) หรือเกิดความหน่วงเกินค่าที่กำหนดในข้อบังคับ

ผู้ออกแบบสามารถทำขั้นตอนนี้ได้โดยใช้ซอฟต์แวร์เช่น M1 ของบริษัท Xilinx หรือผู้ออกแบบจะทำการเชื่อมต่อสัญญาณด้วยตัวเอง (manual layout) ก็ได้ แต่ทางที่ดีควรใช้ซอฟต์แวร์ทำดีกว่าโดยให้ ทำการค้นหาเส้นทางหลายๆ ครั้งเพื่อหาครั้งที่ดีที่สุด นอกจากนั้นการกำหนดข้อบังคับทางเวลา (timing constraints) จะช่วยให้ผลที่ได้จากการทำเชื่อมต่อสัญญาณดีขึ้นได้

2.5.5 ความหน่วงด้านเวลา (Delay)

ในการทำ FPGA นั้นความหน่วงที่เกิดขึ้นเป็นความหน่วงที่เกิดจากการวางตำแหน่ง (layout) ของอุปกรณ์ ซึ่งผู้ออกแบบไม่สามารถเข้าไปแก้ไขได้ แต่สามารถทำให้มีความหน่วงน้อยที่สุดได้ สำหรับความหน่วงที่เกิดขึ้นนั้นแยกได้เป็นสองประเภทคือ ความหน่วงลอจิก (Logic delay) เป็นความหน่วงภายในองค์ประกอบของอุปกรณ์ FPGA เอง เช่นความหน่วงภายใน CLBs และความหน่วงที่เกิดจากการเชื่อมต่อ

สัญญาณ (Routing delay) เป็นความหน่วงที่เกิดจากการเชื่อมต่อสัญญาณระหว่างองค์ประกอบภายในของอุปกรณ์ FPGA โดยปกติแล้วค่าความหน่วงลอจิกไม่ควรเกิน 50 % ของค่าความหน่วงที่ยอมรับได้ เพราะว่าความหน่วงที่เกิดจากการเชื่อมต่อสัญญาณมักจะมีค่ามากกว่าค่าความหน่วงลอจิก ดังนั้นในการวางอุปกรณ์ และเชื่อมต่อสัญญาณ ผู้ออกแบบควรกำหนดข้อบังคับทางเวลาเพื่อให้ซอฟต์แวร์ได้ทำงานอย่างมีประสิทธิภาพเพิ่มขึ้น และเพื่อให้ได้ผลลัพธ์ที่ดีขึ้น

ค่าความหน่วง ที่ได้หลังจากการวางอุปกรณ์ และเชื่อมต่อสัญญาณแล้วเป็นค่าความหน่วงที่ค่อนข้างแน่นอน ซึ่งผู้ออกแบบสามารถทราบได้ว่าโมเดลที่ออกแบบนั้น เป็นไปตามข้อกำหนดหรือไม่

2.5.6 การโปรแกรมอุปกรณ์ FPGA (Configuration)

หลังจากที่โมเดลผ่านขั้นตอนต่างๆ จนกระทั่งผ่านการทำ PPR (Partitioning , Placement & Routing) แล้วนั้น ถึงตอนนี้ก็สามารถที่จะดาวน์โหลด (download) ลงในอุปกรณ์ FPGA ได้แล้ว ในการดาวน์โหลดนี้ ก่อนอื่นต้องแปลงแบบ วงจรรวมที่ได้เป็นข้อมูลวงจร (Configuration data) ซึ่งอยู่ในรูปของบิตสตรีม (bit stream) ก่อน แล้วจึงดาวน์โหลดลงไปเพื่อให้อุปกรณ์ FPGA มีฟังก์ชันการทำงานตามโมเดลที่ผู้ออกแบบต้องการ ซึ่งในขั้นตอนนี้จะใช้วิธีที่แตกต่างกันออกไปสำหรับอุปกรณ์ FPGA ของแต่ละบริษัทผู้ผลิตคือ ในกรณีที่เป็นอุปกรณ์ FPGA ชนิดที่ต้องโปรแกรมโดยวิธี SRAM นั้น ในการใช้งานผู้ออกแบบต้องเก็บข้อมูลวงจร (Configuration data) ไว้ในหน่วยความจำประเภท EPROM หรือ serial PROM ด้วยเพื่อจะได้ใช้งานได้สะดวกขึ้น คือในการใช้งานโมเดลครั้งต่อไปไม่ต้องดาวน์โหลดข้อมูลวงจรจากเครื่องคอมพิวเตอร์อีก เพราะมีข้อมูลวงจรเก็บอยู่ในหน่วยความจำอยู่แล้ว แต่ในกรณีที่อุปกรณ์ FPGA เป็นชนิดที่โปรแกรมโดยใช้วิธี EPROM หรือ Antifuse ก็ไม่จำเป็นต้องมีหน่วยความจำไว้สำหรับเก็บข้อมูลวงจร เพราะว่าอุปกรณ์ FPGA ชนิดนี้เมื่อดาวน์โหลดลงไปแล้ว ข้อมูลที่ดาวน์โหลดลงไปก็ยังคงอยู่ในอุปกรณ์ FPGA และครั้งต่อไปก็ใช้งานโมเดลที่ออกแบบไว้ได้เลย

2.5.7 การจำลองการทำงานของวงจร (Simulation)

ในขั้นตอนนี้เป็นขั้นตอนสำคัญอีกขั้นตอนหนึ่ง เพราะเป็นขั้นตอนที่ผู้ออกแบบตรวจสอบฟังก์ชันการทำงานของโมเดลว่าถูกต้องหรือไม่ มีข้อผิดพลาดตรงไหนเพื่อจะได้ทำการแก้ไขให้ถูกต้อง ในขั้นตอนนี้จะมีซอฟต์แวร์ที่ใช้สำหรับทำการจำลองการทำงานของวงจร ที่ใช้อยู่เช่น ModelSim ของบริษัท Model Technology ในการจำลองการทำงานของวงจร ควรทำทุกครั้งหลังจากที่มีการแก้ไขขั้นตอนหลักเสร็จแล้ว เพื่อจะได้ทราบว่าข้อผิดพลาดของโมเดลเกิดจากขั้นตอนไหนจะได้แก้ไขข้อผิดพลาดตรงนั้นๆ ได้เลย ไม่ต้องมาคอยตรวจหาขั้นตอนที่ทำให้เกิดการผิดพลาด นั่นคือการทำการจำลองการทำงานของวงจร ต้องทำหลังจากการเขียนโค้ด, การสังเคราะห์วงจร และการทำ PPR การจำลองการทำงานหลังจากที่เขียนโค้ดเสร็จแล้วนั้น ผู้ออกแบบสามารถทราบได้แค่โมเดลทำงานถูกต้องหรือไม่เท่านั้น (functional test) แต่ยังไม่สามารถตรวจสอบการทำงานในเชิงเวลาได้ถูกต้อง ในการจำลองการทำงานของวงจรหลังจากที่สังเคราะห์เป็นวงจรแล้วเพื่อตรวจสอบว่าฟังก์ชันการทำงานยังคงถูกต้องอยู่หรือไม่ และค่าความหน่วงที่เกิดขึ้นเป็นไปตามข้อบังคับหรือไม่ มีข้อผิดพลาดหรือไม่ ถ้ามีก็จะได้แก้ไขให้ถูกต้อง ในการจำลองการทำงานของวงจรหลังจากที่ทำการวางอุปกรณ์ และเชื่อมต่อสัญญาณ(Post Layout Simulation) แล้วก็มีความสำคัญเช่นกันเพราะผลที่ได้จากการจำลองการทำงานของวงจรในตอนนี้จะเป็นผลลัพธ์ของโมเดลเลย ซึ่งผู้ออกแบบนอกจากจะตรวจสอบ

ฟังก์ชันการทำงานแล้วยังต้องตรวจสอบคุณสมบัติอื่นๆ เช่น ความหน่วงที่ได้จากการทำ PPR ในรูปแบบค่าความหน่วงมาตรฐาน (Standard Delay Format :SDF) ว่าตรงตามที่กำหนดไว้หรือไม่ หรือตรวจสอบว่าแบบวงจรรวมสามารถใช้งานที่ความถี่สูงสุดเท่าไร นั่นเอง ในการจำลองการทำงานของวงจรควรใช้ ซอฟต์แวร์ตัวเดียวกันตลอดเพื่อจะได้เปรียบเทียบผลที่ได้จากขั้นตอนต่างๆ

2.5.8 ซอฟต์แวร์ FPGA (FPGA Design Tools)

จะเห็นได้ว่าการออกแบบเพื่อทำ FPGA นั้นทำได้สะดวกกว่าการทำ ASIC มากเพราะใช้เวลาน้อยกว่ามากด้วย ส่วนสำคัญที่ใช้ในการทำ FPGA คือ ซอฟต์แวร์ที่ใช้ตั้งแต่เขียนโค้ดอธิบายฮาร์ดแวร์จนกระทั่งดาวน์โหลดลงในอุปกรณ์ FPGA ซึ่งซอฟต์แวร์ที่ต้องเป็น ซอฟต์แวร์ที่ใช้ทำงานต่อเนื่องกันได้ คือ ซอฟต์แวร์ที่ใช้ทำการสังเคราะห์วงจรกับ ซอฟต์แวร์ ที่ใช้ทำ PPR เช่น Leonardo กับ M1 หรือ Synopsys กับ M1 นั่นคือ M1 ต้องสามารถทำ PPR วงจรที่สังเคราะห์โดยซอฟต์แวร์ Leonardo หรือ Synopsys ได้ สำหรับซอฟต์แวร์ที่ใช้ทำการจำลองการทำงานของวงจรมัน ต้องสามารถใช้งานได้ต่อเนื่องกับซอฟต์แวร์ทั้งระบบเพราะโมเดลที่ได้จากการทำขั้นตอนต่างๆ (ด้วยซอฟต์แวร์ต่างๆ) ต้องเอามาจำลองการทำงานได้ และในการจำลองการทำงานของวงจร ควรใช้ ซอฟต์แวร์ตัวเดียวกันตลอดทั้งระบบ เพื่อจะได้เปรียบเทียบผลได้ง่าย ในอดีตซอฟต์แวร์ส่วนใหญ่จะใช้งานอยู่บนคอมพิวเตอร์สมรรถนะสูงอย่างเวิร์กสเตชัน (Workstation) ในปัจจุบันมีการพัฒนาซอฟต์แวร์ที่ใช้งานบนพีซี (PC) มากขึ้น ซึ่งสามารถลดค่าใช้จ่ายในด้านอุปกรณ์คอมพิวเตอร์ ส่วนรายชื่อตัวอย่างซอฟต์แวร์และบริษัทผู้ผลิตที่ใช้ในการออกแบบแต่ละขั้นตอนแสดงดังตารางที่ 2.3

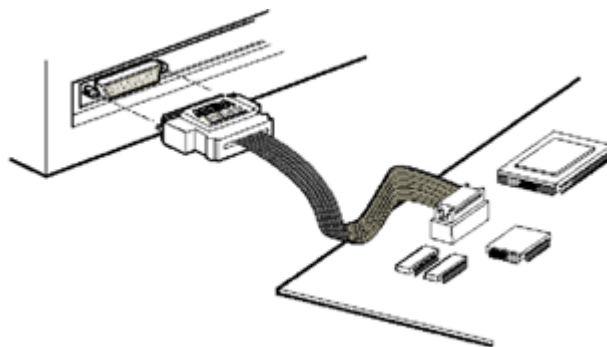
ขั้นตอนการออกแบบ	ซอฟต์แวร์ที่ใช้	ผู้ผลิต
HDL Simulation	ModelSim/VHDL	Model Technology
HDL Synthesis & Optimization	Leonardo	Exemplar logic
Partitioning, Placement and Routing	Foundation	Xilinx

ตารางที่ 2-1 แสดงตัวอย่างรายชื่อซอฟต์แวร์และผู้ผลิตที่ใช้ในการออกแบบแต่ละขั้นตอน

2.6 ปัจจัยที่ทำให้การออกแบบ FPGA ทำได้ง่ายและสะดวกรวดเร็ว

- ผู้ออกแบบไม่จำเป็นต้องทราบถึงโครงสร้างภายในของตัวชิพ เพียงแต่มีความรู้เกี่ยวกับขั้นตอนการออกแบบลอจิกก็เพียงพอแล้ว ต่างกับการใช้ไมโครโปรเซสเซอร์ซึ่งจำเป็นต้องศึกษาโครงสร้างภายในรวมถึง ภาษา Assembly ของไมโครโปรเซสเซอร์ตัวนั้นด้วย
- มีการออกแบบโดยใช้ภาษาในการอธิบายการทำงานของวงจร หรือ HDL (Hardware Description Language) เป็นเครื่องมือในการออกแบบ ซึ่งเป็นวิธีการที่มีความยืดหยุ่นสูง ทำได้รวดเร็ว และไม่จำเป็นต้องทราบถึงลักษณะของวงจรที่ต้องการว่าจะเชื่อมต่อกันอย่างไร เพียงแต่กำหนดลักษณะการทำงานให้มัน จากนั้นตัวซอฟต์แวร์จะทำ Synthesis and Optimize ให้ทั้งหมด นอกจากนี้ภาษาที่ใช้ยังเป็นมาตรฐาน เดียวกันสามารถใช้ได้กับชิพทุกตัวและทุกบริษัท
- การโปรแกรมสามารถทำได้เองและใช้เวลาไม่นาน เพียงแค่ส่งข้อมูลผ่านสายดาวน์โหลดทางพอร์ตของ คอมพิวเตอร์ก็สามารถโปรแกรมตัวชิพขณะที่อยู่ในระบบได้ โดยไม่จำเป็นต้องถอดมาโปรแกรม

ข้างนอก ดังรูปที่ 2-13 และที่สำคัญสามารถโปรแกรมได้หลายครั้ง จึงทำให้ง่ายในการแก้ไขและพัฒนาโดยไม่ต้องเสียค่าใช้จ่ายเพิ่มแต่อย่างใด



รูปที่ 2-13 การโปรแกรมลงในชิพ

การออกแบบโดยใช้ภาษาอธิบายพฤติกรรมของฮาร์ดแวร์

ในการออกแบบวงจรดิจิทัลนั้นสามารถทำได้โดยการวาดวงจร (Schematic) หรือใช้ภาษาอธิบายพฤติกรรม (Hardware Description Language) ของฮาร์ดแวร์ จากที่ได้กล่าวไปแล้วในข้างต้น ในกรณีของการออกแบบวงจรด้วย ASIC ชนิด Full Custom ผู้ออกแบบจะต้องเขียนวงจรด้วย Schematic จากนั้นจะนำวงจรที่ออกแบบไว้ไปทำการจำลองการทำงาน (Simulate) ซึ่งหากผลออกมาเป็นที่พอใจก็จะต้อง Layout เป็นชั้นสาร และในการออกแบบ ASIC ชนิดนี้ผู้ออกแบบจำเป็นต้องทราบถึงเทคโนโลยีที่ใช้ในการสร้างด้วย หลังจากได้ Layout ที่สมบูรณ์แล้วจึงจะส่งไปเข้ากระบวนการสร้างไอซีหรือ Fabrication เพื่อสร้างเป็นชิพไอซีออกมา แต่ในการออกแบบวงจรด้วย FPGA โดยการใช้ Schematic หรือใช้ภาษาอธิบายการทำงานของวงจรจะทำได้สะดวกกว่า เนื่องจากวิธีการนี้ผู้ออกแบบไม่จำเป็นต้องคำนึงถึงเทคโนโลยีที่จะใช้สร้างไอซีและที่สำคัญ การออกแบบโดยวิธีนี้สามารถแก้ไขโมเดล (Model) หรือเปลี่ยนแปลงเทคโนโลยีได้สะดวกกว่า เพราะไม่ต้องวาดวงจรใหม่ นั่นคือการออกแบบโดยใช้ภาษาอธิบายฮาร์ดแวร์ จะทำให้โมเดลที่ได้ไม่ขึ้นกับเทคโนโลยีสำหรับภาษาที่ใช้ สำหรับอธิบายพฤติกรรมของฮาร์ดแวร์ที่ใช้กันก็มี VHDL, AHDL และ Verilog เป็นต้น

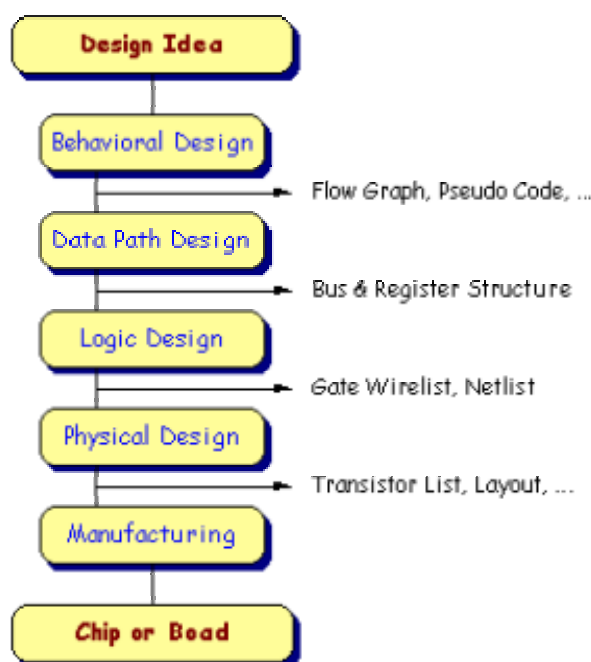
2.7 การเขียนภาษา VHDL

ความซับซ้อนและขนาดของระบบดิจิทัลในปัจจุบันได้เพิ่มมากขึ้นทุกขณะ ส่งผลให้มีการนำคอมพิวเตอร์เพื่อช่วยในการออกแบบหรือ CAD มาใช้ในขบวนการออกแบบฮาร์ดแวร์เพิ่มขึ้นเช่นกัน อีกทั้งอุปกรณ์และวิธีการออกแบบใหม่ๆ ก็ถูกพัฒนาขึ้นมาเพื่อช่วยอำนวยความสะดวกให้กับนักออกแบบมากขึ้นด้วย สำหรับภาษาบรรยายอุปกรณ์ฮาร์ดแวร์ (HDL : Hardware Description Language) ก็เป็นเครื่องมืออย่างหนึ่งที่ได้รับการพัฒนาอย่างต่อเนื่อง เพื่อช่วยให้การปรับปรุงขบวนการออกแบบระบบดิจิทัลเป็นไปอย่างมีประสิทธิภาพ

2.7.1 การออกแบบระบบดิจิทัล

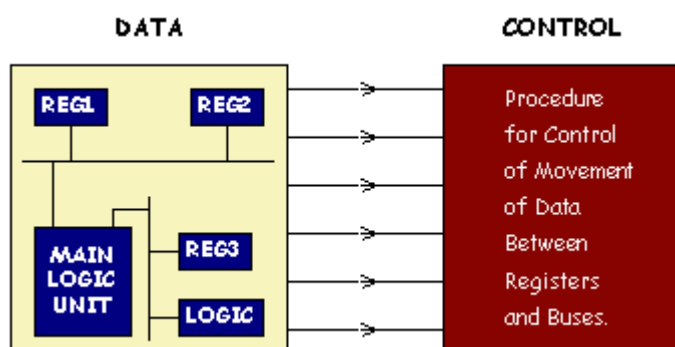
ในการออกแบบระบบดิจิทัล เริ่มตั้งแต่การกำหนดแนวความคิดเบื้องต้นจนกระทั่งได้ออกมาเป็นอุปกรณ์ฮาร์ดแวร์ ที่ใช้งานได้จะต้องผ่านขั้นตอนต่างๆ มากมาย และในแต่ละขั้นตอนผู้ออกแบบจะต้องตรวจสอบผลลัพธ์ในแต่ละขั้น ก่อนเข้าสู่กระบวนการออกแบบในขั้นต่อไป รูปที่ 2-14 แสดงขั้นตอนปกติที่ใช้ในการออกแบบระบบดิจิทัลทั่วไป ขั้นแรกผู้ออกแบบจะกำหนดแนวความคิดในการออกแบบแล้วทำการ

พัฒนาให้สามารถนำมาใช้ได้อย่างสมบูรณ์ ซึ่งภายในขั้นตอนนี้ออกแบบจำเป็นต้องสร้างรูปแบบระบบในเชิงพฤติกรรมขึ้นมาตรวจสอบซึ่งอาจจะเป็นผังงานแสดงแบบหรือ รหัสคำสั่งเทียม (Pseudo code) ก็ได้



รูปที่ 2-14 แสดงขั้นตอนการออกแบบระบบดิจิทัล

ขั้นตอนต่อไปเป็นการออกแบบระบบเส้นทางของข้อมูล ผู้ออกแบบจะกำหนดส่วนประกอบของรีจิสเตอร์และวงจรถลอจิก ที่จำเป็นทั้งหมดเพื่อนำมาประกอบเป็นระบบที่สมบูรณ์ โดยแต่ละองค์ประกอบสามารถเชื่อมต่อกันด้วยบัสหนึ่งหรือสอง ทิศทาง (Unidirectional or Bidirectional Bus) ส่วนกระบวนการในการควบคุมการเคลื่อนย้ายข้อมูลระหว่าง รีจิสเตอร์และวงจรถลอจิกจะขึ้นอยู่กับพฤติกรรมของระบบที่กำหนดไว้ดังรูปที่ 2-15



รูปที่ 2-15 การออกแบบระบบเส้นทางของข้อมูล

ขั้นตอนถัดมาเป็นการออกแบบวงจรถลอจิก ซึ่งจะเกี่ยวข้องกับการนำเกทดิจิทัลพื้นฐานและฟลิปฟล็อป (flip-flop) มาประกอบเป็นอุปกรณ์ย่อยต่างๆ เช่น รีจิสเตอร์เก็บข้อมูล บัสวงจรถลอจิก และส่วนควบคุมฮาร์ดแวร์ ซึ่งผลลัพธ์ ที่ได้ในขั้นตอนนี้จะเป็นเครือข่ายของการโยงใยระหว่างเกทและ ฟลิปฟล็อป

นั่นเองการออกแบบในขั้นตอนถัดไป เป็นการเปลี่ยนเครือข่ายการโยงใยที่ได้จากขั้นตอนที่แล้วให้เป็นลำดับของทรานซิสเตอร์ (Transistor List) และ Layout ซึ่งขั้นตอนนี้จะเกี่ยวข้องโดยตรงกับการจัดวางทรานซิสเตอร์หรือไลบรารีเซลล์เพื่อ แทนเกทและ ฟลิปฟล็อปต่างๆและในขั้นตอนสุดท้ายจะเป็นการส่งระบบที่ออกแบบไว้ไปทำการเจือสารที่โรงงานเพื่อผลิตออกมาเป็น วงจรรวมในที่สุด

2.7.2 ประวัติความเป็นมาของภาษา VHDL

VHDL ย่อมาจากคำว่า VHSIC Hardware Description Language (VHSIC : Very High Speed Integrated Circuit) เป็นภาษาโปรแกรมระดับสูง (High Level Language) ที่ใช้สำหรับการออกแบบฮาร์ดแวร์ในระบบดิจิทัล ตัวของภาษาสามารถบรรยายพฤติกรรมการทำงานในรูปของลำดับชั้น (Hierarchy) และสามารถเขียนได้หลายรูปแบบ ด้วยเหตุผลนี้จึงทำให้ภาษา VHDL เป็นเครื่องมือที่ใช้ออกแบบตั้งแต่ขั้นตอนบนสุด คือ แนวความคิดที่จะแก้ปัญหา ลงไปที่ละขั้นจนถึงขั้นตอนของการสร้างวงจรจริง และตัวภาษาก็เปิดโอกาสให้วิศวกร ได้พัฒนาและจำลองการทำงานของรูปแบบฟังก์ชันการทำงานของวงจรอย่างสังเขป โดยยังไม่ต้องคำนึงถึงรายละเอียดเกี่ยวกับโครงสร้างวงจรจริง นอกจากนั้น VHDL ยังเป็นภาษาที่สนับสนุนลักษณะต่างๆ ของระบบดิจิทัลที่มี ความซับซ้อนได้ทั้งหมด ดังนั้น VHDL จึงเป็นภาษาที่น่าสนใจในการศึกษาและนำไปใช้งานเป็นอย่างยิ่ง วิวัฒนาการของภาษา VHDL เริ่มต้นประมาณปี ค.ศ. 1981 เมื่อกระทรวงกลาโหมสหรัฐอเมริกา หรือ DoD (Department of Defense) ได้พยายามปรับปรุงอุปกรณ์อิเล็กทรอนิกส์และคอมพิวเตอร์ที่ใช้ในกิจการทางทหาร ให้มีความทันสมัยมากขึ้น ประกอบกับเทคโนโลยีทางด้านไมโครอิเล็กทรอนิกส์มีการพัฒนาไปอย่างรวดเร็วดังจะเห็นได้ จากการนำวงจรดิจิทัลหลายๆ วงจรมาทำการผลิตอยู่บนแผ่นซิลิกอนที่มีพื้นที่เพียง 1 - 2 ตารางเซนติเมตรเท่านั้น ซึ่ง เป็นผลให้ประสิทธิภาพในการทำงานของวงจรสูงขึ้นตลอดจนความน่าเชื่อถือ ในการทำงานและความคงทนต่อสภาพ แวดล้อมสูง แต่เนื่องจากในขณะนั้นขั้นตอนของการออกแบบ การผลิต และการตรวจสอบวงจรต้นแบบ เป็นขบวนการที่ต้องใช้วิศวกร และเวลาในดำเนินการมาก ฉะนั้นทาง DoD จึงจัดตั้งโครงการขึ้นมาเพื่อศึกษาวิธีการที่ช่วยในการพัฒนา วงจรอิเล็กทรอนิกส์ โดยเฉพาะอย่างยิ่งวงจรระบบดิจิทัล ให้สามารถนำไปผลิตได้เร็วขึ้น ซึ่งโครงการดังกล่าวมีชื่อว่า "Very High Speed Integrated Circuits" หรือ VHSIC โดยในระยะแรกนั้นโครงการนี้ถือเป็นความลับทางด้านความมั่นคงของประเทศ และอยู่ภายใต้ความควบคุมดูแลของ United States International Traffic and Arms Regulations (ITAR) สำหรับมาตรฐานของภาษาที่ใช้บรรยาย พฤติกรรมวงจรหรือฮาร์ดแวร์ของระบบ สำหรับโครงการ VHSIC ที่ DoD ได้ให้ไว้สามารถสรุปได้ดังนี้

- ต้องเป็นภาษาที่นำไปเขียนรูปแบบระบบดิจิทัล และมีคุณสมบัติที่สามารถเข้าใจได้ทั้งมนุษย์และเครื่อง คอมพิวเตอร์โดยไม่ต้องมีการแปลหรือเปลี่ยนแปลงอีก
- สามารถนำไปใช้เป็นเอกสารประกอบโครงการได้
- ต้องเป็นภาษาที่เขียนขึ้นสำหรับใช้จำลองการทำงานของวงจร

ฉะนั้นภาษาดังกล่าวนี้นี้จึงจัดเป็นภาษาโปรแกรมระดับสูง เช่นเดียวกับภาษาปาสคาล หรือภาษาซี ซึ่งในทางวิศวกรรม ภาษาที่ใช้ในการออกแบบฮาร์ดแวร์นี้เรียกว่า "Hardware Description Language" หรือ HDL ในตอนเริ่มแรกนั้น DoD ได้มอบหมายให้บริษัทไอบีเอ็ม เท็กซัสอินสตรูเมนต์ และอินเตอร์เมทริกซ์ เป็นผู้ศึกษาและพัฒนา โครงการ ซึ่งการดำเนินงานเป็นไปอย่างต่อเนื่อง จนกระทั่งในปี ค.ศ.1985 ทาง ITAR ได้ยกเลิก

ข้อจำกัดในการถ่ายทอด เทคโนโลยีทางทหารออกจากโครงการนี้ ดังนั้นภาษา VHDL จึงเริ่มเป็นที่รู้จักกันโดยทั่วไป และประมาณปี ค.ศ. 1987 IEEE ได้ทำการกำหนดมาตรฐานของภาษานี้เป็น IEEE 1076-1987 และมีชื่อเรียกว่า VHDL ซึ่งมาตรฐานนี้ได้รับการปรับปรุงจนเป็นมาตรฐาน IEEE 1076-1993 หรือ VHDL 1993 เนื่องจากในขณะนั้น DoD เป็นลูกค้ารายใหญ่ ของอุตสาหกรรมอิเล็กทรอนิกส์และคอมพิวเตอร์ ดังนั้นจึงมีผู้รับโครงการต่างๆ จาก DoD ไปดำเนินการวิจัยและพัฒนา เป็นจำนวนมาก และเพื่อให้ทุกโครงการอยู่ในมาตรฐานเดียวกันหมด ดังนั้นทาง DoD จึงได้กำหนดว่า ทุกๆ โครงการต้อง เขียนอยู่ในรูปของภาษา VHDL เท่านั้น ซึ่งทำให้ DoD สามารถนำโครงการเหล่านี้ไปจำลองกับเครื่องคอมพิวเตอร์ได้ หลายๆ ระบบ

ข้อกำหนด DoD ได้ตั้งข้อกำหนดสำหรับภาษา VHDL ในเดือนมกราคมปี ค.ศ.1983 ไว้ดังนี้

- **ลักษณะทั่วไป**

DoD ได้กำหนดให้ VHDL เป็นภาษาสำหรับการออกแบบและบรรยายของฮาร์ดแวร์ ซึ่งหมายถึงความสามารถ ในการอธิบายและออกแบบในระดับสูง การจำลอง (Simulation) การสังเคราะห์ (Synthesis) และการทดสอบ (Testing) นอกจากนั้น VHDL ยังถูกกำหนดไว้สำหรับการบรรยายฮาร์ดแวร์ตั้งแต่ระดับบนซึ่งก็คือระบบจนถึง ระดับเกทอีกด้วย เนื่องจากการทำงานของระบบดิจิทัลนั้น ทุกๆ องค์ประกอบภายในระบบไม่ว่าเล็กหรือใหญ่ จะทำงานไปพร้อมๆ กัน ซึ่งในเรื่องของความพร้อมเพรียงในการทำงานนี้ก็ถือเป็นข้อกำหนดที่สำคัญอย่างหนึ่งของ VHDL ด้วยเช่นกัน (สำหรับในภาษาที่ใช้ในการบรรยายฮาร์ดแวร์นั้นความพร้อมเพรียงจะหมายถึงทุกๆ คำสั่ง องค์ประกอบ เกทหรือวงจรลอจิกจะถูกนำมาปฏิบัติทั้งหมด ดังนั้นในที่สุดแล้วก็จะดูเหมือนว่าได้มีการปฏิบัติไป พร้อมๆ กัน)

- **สนับสนุนการออกแบบแบบลำดับขั้น**

การออกแบบแบบลำดับขั้นเป็นลักษณะที่สำคัญอย่างหนึ่งสำหรับการออกแบบระบบที่มีหลายๆ ระดับ โดยในการ ออกแบบจะประกอบด้วยส่วนการบรรยายการเชื่อมต่อ และส่วนการบรรยายหน้าที่การทำงาน ซึ่งหน้าที่การทำงาน ของระบบสามารถกำหนดได้ด้วยตัวเอง หรืออาจถูกกำหนดโดยโครงสร้างที่ประกอบด้วยองค์ประกอบย่อยๆ ลง ไปได้เช่นกัน แต่ที่ระดับล่างสุด องค์ประกอบต้องถูกบรรยายหน้าที่การทำงานด้วยตัวมันเอง และไม่สามารถกำหนด การทำงานโดยลักษณะแบบโครงสร้างได้

- **ไลบรารี**

VHDL ได้สนับสนุนการมีไลบรารีเพื่อระบบการจัดการที่ดี ผู้ออกแบบสามารถกำหนดลักษณะและการทำงานของ อุปกรณ์พื้นฐานไว้ในระบบไลบรารี หรือจะใช้ไลบรารีที่ระบบได้จัดเตรียมไว้แล้วก็ได้ โมเดลและการบรรยายที่ถูก ต้องควรจัดเก็บไว้ในไลบรารีหลังจากที่ได้ผ่านการคอมไพล์เรียบร้อยแล้วเพื่อให้ผู้ออกแบบคนอื่นๆ สามารถนำไป ใช้ได้ด้วย

- **ลำดับคำสั่ง**

แม้ว่าการปฏิบัติคำสั่งหรือกระบวนการโดยพร้อมเพรียงกันจะเป็นคุณสมบัติที่สำคัญของ VHDL ก็ตาม ตัวภาษา เองก็ยังมีการจัดเตรียมลักษณะการควบคุมแบบลำดับคำสั่งไว้ให้ด้วย เมื่อผู้ออกแบบได้กำหนดหน้าที่และองค์ประกอบ ที่ทำงานพร้อมกันของระบบไว้เรียบร้อยแล้ว ผู้ออกแบบยังสามารถบรรยายหน้าที่การทำงานซึ่งเป็นรายละเอียดภายใน ของแต่ละองค์ประกอบได้ในลักษณะเดียวกับการเขียนโปรแกรมที่ประกอบด้วยโครงสร้างแบบ case, if - then - else และ loop ทั่วๆ ไปได้ การบรรยายแบบลำดับคำสั่งทำให้การ

ออกแบบหน้าที่การทำงานของอุปกรณ์กระทำได้ สะดวกและง่ายขึ้น อย่างไรก็ตามโครงสร้างทั้งหมดของ VHDL ก็ยังคงเป็นการทำงานแบบพร้อมเพรียงกันเช่นเดิม

- **การกำหนดคุณสมบัติ**

นอกจากการกำหนดอินพุตและเอาต์พุตแล้ว เงื่อนไขอื่นๆ ก็มีผลต่อการปฏิบัติหน้าที่ของอุปกรณ์ฮาร์ดแวร์ด้วยเช่นกัน โดยสิ่งนี้รวมถึงสภาพแวดล้อมและลักษณะทางกายภาพของอุปกรณ์นั้นๆ ด้วย ซึ่งภาษาสำหรับการออกแบบที่ดีควร ให้ผู้ออกแบบกำหนดคุณสมบัติของอุปกรณ์ที่ใช้ได้ด้วย เช่น สามารถกำหนดขนาด ลักษณะทางกายภาพเวลา โหลด และเงื่อนไขทางสภาพแวดล้อมอื่นๆ ซึ่งความสามารถในการกำหนดคุณสมบัตินี้ก็เป็นส่วนหนึ่งที่มีอยู่ในภาษา VHDL ด้วยเช่นกัน

- **ชนิดของข้อมูล**

VHDL สามารถกำหนดชนิดของข้อมูลไม่เพียงแต่ชนิด BIT และ BOOLEAN เท่านั้น แต่ยังสามารถกำหนดชนิด ของข้อมูลเป็นจำนวนเต็ม จำนวนจริง จุดทศนิยม และชนิดลำดับการนับ (Enumerate Type) หรือแม้แต่ชนิดของ ข้อมูลที่ผู้ออกแบบกำหนดขึ้นมาเองก็ได้

- **โปรแกรมย่อย**

ความสามารถในการใช้ฟังก์ชันและโพรซีเจอร์ (Procedure) ก็เป็นข้อกำหนดอีกอย่างหนึ่งใน VHDL ซึ่งผู้ออกแบบ สามารถนำโปรแกรมย่อยมาใช้ในการเปลี่ยนแปลงชนิดของข้อมูล การกำหนดหน่วยของลอจิก การกำหนดตัวกระทำต่างๆ หรือหน้าที่อื่นๆ ตามที่ต้องการได้เช่นเดียวกับการเขียนโปรแกรมทั่วไป

- **การควบคุมเวลา**

VHDL อนุญาตให้ผู้ออกแบบสามารถกำหนดเวลาในการส่งผ่านข้อมูลหรือสัญญาณได้ตามต้องการ การตรวจสอบ การออกแบบเกทหรือการหน่วงเวลาก็สามารถกระทำได้โดยการกำหนดช่วงเวลาที่น่านอนหรือกำหนดให้มีการรอคอย เหตุการณ์ (Event) นอกจากนี้ก็ยังสามารถกำหนดรูปแบบของสัญญาณนาฬิกาได้อีกด้วย

- **การกำหนดแบบโครงสร้าง**

การกำหนดโครงสร้างขององค์ประกอบต่างๆ สามารถกระทำได้ในทุกระดับของการออกแบบ โดยการกำหนดโครงสร้างขององค์ประกอบรวมที่เกิดจากองค์ประกอบย่อยซึ่งแตกต่างกันหรือ เหมือนกันก็เป็นข้อกำหนดอย่างหนึ่งของ VHDL เช่นกัน

2.7.3 องค์ประกอบพื้นฐานของ VHDL

รูปแบบพื้นฐานที่ใช้ในการบรรยายถึงองค์ประกอบของ VHDL จะประกอบไปด้วยส่วนกำหนดการเชื่อมต่อ (Interface) และส่วนกำหนดลักษณะเชิงสถาปัตยกรรม (Architecture) ดังแสดงในรูปที่ 2-16 โดยในการบรรยายการเชื่อมต่อจะขึ้น ต้นด้วยคำว่า ENTITY แล้วตามด้วยชื่อขององค์ประกอบจากนั้นตามด้วยคำว่า IS และถัดมาจะเป็นการบรรยายถึงพอร์ต์ การติดต่อ อินพุต - เอาต์พุต ขององค์ประกอบ ส่วนลักษณะภายนอกอื่น ๆ เช่น เวลา อุณหภูมิก็สามารถรวมเข้าไปในส่วนนี้ ได้เช่นกัน ในส่วนของการกำหนดลักษณะเชิงสถาปัตยกรรมจะขึ้นต้นด้วยคำว่า ARCHITECTURE ซึ่งเป็นส่วนที่ใช้ บรรยายหน้าที่การทำงานขององค์ประกอบ โดยหน้าที่การทำงานนี้จะขึ้นอยู่กับสัญญาณอินพุต เอาต์พุตและพารามิเตอร์ อื่นๆ ที่ได้กำหนด

ไว้ในส่วนของการเชื่อมต่อดังรูปที่ 2-16 และสำหรับการบรรยายหน้าที่ขององค์ประกอบจะเริ่มต้นหลังจาก คำว่า BEGIN เป็นต้นไป

```

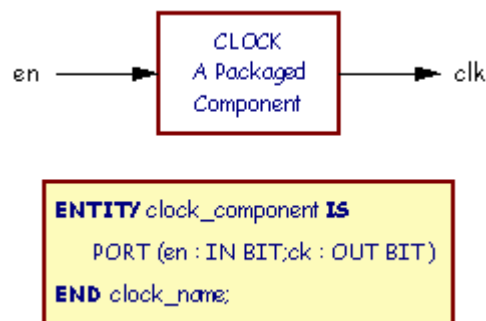
ENTITY component_name IS
    Input and output ports
    Physical and other parameters
END [component_name] ;

ARCHITECTURE identifier OF component_name IS
    [declaration]
BEGIN
    specification of the functionality of the component
    in terms of its input lines and as influenced
    by physical and other parameters
END [identifier];
  
```

รูปที่ 2-16 การกำหนดการเชื่อมต่อและสถาปัตยกรรม

- การกำหนดการเชื่อมต่อ

การกำหนดการเชื่อมต่อเป็นระดับบนสุดของการออกแบบ โดยในระดับนี้ต้องกำหนดพอร์ตสำหรับการติดต่อกับองค์ประกอบ ภายนอกอื่นๆ ดังตัวอย่างในรูปที่ 2-17 ซึ่งเป็นบล็อกไดอะแกรม และการบรรยายการเชื่อมต่อขององค์ประกอบสำหรับตัวจ่าย สัญญาณนาฬิกา ในบรรทัดแรกของการบรรยายการเชื่อมต่อเป็นการกำหนดชื่อขององค์ประกอบซึ่งกำหนดเป็น clock_component ตามด้วยคำว่า PORT และชื่อของพอร์ตอยู่ในวงเล็บ ส่วน IN และ OUT เป็นการกำหนด โหนดของสัญญาณให้เป็นอินพุตหรือเอาต์พุต และ BIT เป็นการแสดงชนิดของข้อมูล



รูปที่ 2-17 บล็อกไดอะแกรมและการบรรยายการเชื่อมต่อของ clock_component

- การกำหนดรูปแบบการบรรยาย

หน้าที่การทำงานขององค์ประกอบจะถูกบรรยายภายในส่วนนี้ ซึ่งในการบรรยายสามารถกำหนดค่าของสัญญาณ เอาต์พุตในเทอมของอินพุตหรือในรูปขององค์ประกอบอื่นๆ หรือทั้งสองอย่างรวมกันก็ได้ ดังตัวอย่างการบรรยายของ clock_component ในรูปที่ 2-18 ซึ่งเป็นการบรรยายในเชิงพฤติกรรม โดยมี en เป็นอินพุตและ ck เป็นเอาต์พุต PROCESS เป็นคำที่ใช้ในการเริ่มต้นสำหรับการบรรยายในเชิงพฤติกรรม และภายในโปรเซสกำหนดให้ periodic เป็นตัวแปรที่มีค่าเริ่มต้นเป็น "0" ถ้าสัญญาณ en มีค่าเป็น "1" จะทำให้ตัว

แปร periodic ถูกคอมพลิเมนต์ (complement) และส่งค่าให้กับ ck ซึ่งเป็นสัญญาณเอาต์พุต และสำหรับคำสั่ง WAIT จะเป็นการกำหนดให้สัญญาณมีคาบเวลาเท่ากับ 1 ไมโครวินาที

```

ARCHITECTURE behavioral OF clock_component IS

BEGIN
  PROCESS
    VARIABLE periodic : BIT := '0';
  BEGIN
    IF en='1' THEN
      periodic := Not periodic;
    END IF;
    ck <= periodic;
    WAIT FOR 1 US;
  END PROCESS;
ENDbehavioral;
  
```

รูปที่ 2-18 การบรรยายเชิงพฤติกรรมของ clock _ component

- หน่วยการออกแบบแพ็คเกจ

ข้อมูลต่างๆ ตลอดจนโปรแกรมย่อย ที่เป็นประโยชน์ต่อการเขียนรูปแบบการบรรยายระบบดิจิทัล สามารถเก็บไว้ใน ส่วนของแพ็คเกจ ซึ่งหน่วยการออกแบบต่างๆ เช่น หน่วยการออกแบบ Entity หน่วยการออกแบบสถาปัตยกรรมหรือ หน่วยการออกแบบแพ็คเกจอื่นๆ สามารถเรียกข้อมูลเหล่านี้ไปใช้ได้ นอกจากนั้น สิ่งที่นิยมทำกันมากคือการนำรูปแบบ มาตรฐานต่างๆ เช่น อุปกรณ์มาตรฐาน (เช่น ไอซีตระกูล 74XX เป็นต้น) มาเก็บไว้ในรูปของแพ็คเกจ ที่ทุกคนสามารถ เข้าถึงได้ ตามปกติแล้วแพ็คเกจจะแบ่งออกเป็น 2 ส่วนคือ การประกาศแพ็คเกจ (Package declaration)และ ส่วนของบอดี้แพ็คเกจ (Package body) เนื่องจาก แพ็คเกจถูกสร้างขึ้นเป็นส่วนแยกต่างหากออกจากรูปแบบที่ กำลังเขียนอยู่ ฉะนั้นการที่นำแพ็คเกจไปใช้นั้นจะต้องมีการ เชื่อมโยงหรืออ้างอิงเสียก่อน ซึ่งในภาษา VHDL สามารถ กระทำได้ด้วยชุดคำสั่ง USE

ก) PACKAGE DECLARATION

ส่วนที่มีความสำคัญที่สุดของแพ็คเกจ (ถ้ามองในแง่ของการนำไปใช้จากภายนอก) ได้แก่ส่วนการ ประกาศแพ็คเกจ เนื่องจากเป็นส่วนที่ใช้กำหนดชื่อของสิ่งที่ประกาศอยู่ภายในแพ็คเกจ สำหรับนำไปใช้ ภายนอกตัวของแพ็คเกจเอง ถ้ามี การประกาศสิ่งใดๆ ในส่วนของส่วนบอดี้แพ็คเกจ แต่ไม่ถูกประกาศในส่วน การประกาศแพ็คเกจจะทำให้ค่าและพฤติกรรมไม่สามารถนำไปใช้งานในส่วนนอกได้ซึ่งเปรียบเทียบกับสิ่งที่ ประกาศไว้ในส่วนของการประกาศ Entity คือ จุดเชื่อมต่อ หรือ พอร์ต ที่มีหน้าที่ติดต่อกับโลกภายนอก ฉะนั้นโดยทั่วไปแล้วแพ็คเกจสามารถสร้างขึ้นได้โดยไม่จำเป็น ต้องมีส่วนบอดี้ และยังสามารถนำไปใช้งาน จากรูปแบบภายนอกได้เช่น ใช้สำหรับประกาศ ชนิด (Type) หรือสัญญาณ เช่นเดียวกับ ส่วนบอดี้แพ็คเกจที่ไม่ จำเป็นต้องมี ส่วนของการประกาศแพ็คเกจ แต่แพ็คเกจนั้นจะไม่สามารถนำไปใช้จาก รูปแบบอื่นได้


```
PACKAGE package_name IS
    Package_declarative_part
END package_name;
```

รูปที่ 2-19 โครงสร้างทั่วไปของส่วนการประกาศแพ็คเกจ

ข) PACKAGE BODY

โครงสร้างซึ่งประกอบด้วยลำดับคำสั่งที่ใช้บรรยายฟังก์ชันการทำงานของโปรแกรมย่อยทั้งหลาย ซึ่งชื่อของโปรแกรมย่อยนั้นๆ ได้ถูกประกาศไปแล้วในส่วนของการประกาศแพ็คเกจ จะถูกเก็บไว้ในส่วนของบอดีแพ็คเกจ ทั้งนี้รวมถึง การกำหนดค่าที่ต่างๆ อันได้แก่ค่าคงที่ที่ถูกประกาศชื่อไว้ก่อนในส่วนของการประกาศแพ็คเกจ และถูกกำหนดค่าใน ส่วนของบอดีแพ็คเกจ ฉะนั้นในส่วนของบอดีแพ็คเกจจึงไม่จำเป็นต้องมี ถ้าในส่วนของการประกาศแพ็คเกจไม่มีการ ประกาศชื่อที่เป็นโปรแกรมย่อย หรือค่าคงที่ การเขียนบอดีแพ็คเกจนั้นจะเป็นไปตามกฎเกณฑ์ดังแสดงในรูปที่ 2-20

```
PACKAGE BODY package_name IS
    declarative part
END package_name;
```

รูปที่ 2-20 โครงสร้างของบอดีแพ็คเกจ

• หน่วยการออกแบบ Configuration

ดังที่ทราบกันแล้วว่าระบบดิจิทัลรูปแบบหนึ่งไม่ว่าจะเป็นอะไรก็ตาม จะสามารถมีหน่วยการออกแบบ Entity ได้ เพียงหนึ่งเดียวเท่านั้น ซึ่งในหน่วยการออกแบบ Entity หนึ่งหน่วยนี้อาจจะมีสถาปัตยกรรมที่เป็นหน่วยรองได้หลาย หน่วย ดังนั้นจะต้องมีหน่วยการออกแบบ Configuration มาเพื่อกำหนดการใช้ Configuration ของการประกอบ Entity กับหน่วยการออกแบบสถาปัตยกรรมหน่วยใดๆ เข้าด้วยกัน

```
CONFIGURATION identifier OF entity_name IS
    Configuration_declarative_part
END ;
```

รูปที่ 2-21 สร้างโดยทั่วไปของหน่วยการออกแบบโครงแบบ

• โปรแกรมย่อย

การใช้ฟังก์ชันและโพรซีเจอร์ใน VHDL เปรียบได้กับการใช้โปรแกรมย่อยในการเขียนโปรแกรมภาษาชั้นสูงต่างๆ ไปค่าที่ถูกส่งกลับหรือถูกเปลี่ยนแปลงโดยโปรแกรมย่อยอาจจะมีหรือไม่มีผลต่อฮาร์ดแวร์โดยตรงก็ได้ เช่นถ้าใช้ฟังก์ชัน แทนการกระทำในสมการบูลีนก็จะมีผลต่อวงจรลอจิกจริงๆ ในขณะที่ถ้าใช้โปรแกรมย่อยในการเปลี่ยนชนิดของข้อมูล หรือในการคำนวณค่าการหน่วงเวลาแล้วก็จะไม่มีผลต่อโครงสร้างของฮาร์ดแวร์ รูปที่ 2-22 แสดงการใช้โพรซีเจอร์ เพื่อเปลี่ยนข้อมูลชนิด 8 บิตเป็นค่าจำนวนเต็ม และรูปที่ 2-23 แสดงการใช้ฟังก์ชันโดยกำหนดให้ X เป็นตัวแปรชนิด บิตแทนการกระทำในสมการบูลีน

```

TYPE byte IS ARRAY (7 DOWNTO 0) OF BIT;
...
PROCEDURE byte_to_integer (ib : IN byte; oi : OUT INTEGER) IS
    VARIABLE result: INTEGER := 0;
BEGIN
    FOR i IN 0 TO 7 LOOP
        IF ib(i) = '1' THEN
            result := result + 2**i;
        END IF;
    END LOOP;
    oi := result;
END byte_to_integer

```

รูปที่ 2-22 การใช้โพรซีเจอร์

```

FUNCTION f (a, b, c: BIT) RETURN BIT IS
    VARIABLE x: BIT;
BEGIN
    x := ((NOT a) AND (NOT b) AND c);
    RETURN x;
END f;

```

รูปที่ 2-23 การใช้ฟังก์ชัน

- โอเปอเรเตอร์

การบรรยายเชิงพฤติกรรมในภาษา VHDL มีตัวดำเนินการหรือโอเปอเรเตอร์ทางลอจิกและคณิตศาสตร์เช่นเดียวกับภาษาซอฟต์แวร์ทั่วไปดังรูปที่ 2-24

PREDEFIND OPERATORS	
LOGICAL OPERATORS :	NOT AND OR NAND NOR XOR
OPERAND TYPE :	BIT BOOLEAN
RESULT TYPE :	BIT BOOLEAN
RELATIONAL OPERATORS :	= /= < <= > >=
OPERAND TYPE :	any type
RESULT TYPE :	Boolean
ARITHMETIC OPERATORS :	+ - * / ** MOD REM ABS
OPERAND TYPE :	INTEGER REAL Physical
RESULT TYPE :	INTEGER REAL Physical
CONCATENATION OPERATOR :	&
OPERAND TYPE :	ARRAY of any type
RESULT TYPE :	array of any type
RESULT TYPE :	array of any type

รูปที่ 2-24 ตัวดำเนินการใน VHDL

- เวลาและความพร้อมเพรียง

ในวงจรอิเล็กทรอนิกส์ทุกๆ ตัวจะอยู่ในสภาพเตรียมพร้อมเสมอ (Always Active) และจะมีเรื่องของเวลา เข้ามาเกี่ยวข้องในทุกๆ เหตุการณ์ที่เกิดขึ้นเสมอ VHDL เป็นภาษาที่ได้รับการออกแบบมาเพื่อให้สามารถบรรยายรูปแบบและการพ้องกันของเวลาสำหรับการทำงานของอุปกรณ์ได้อย่างถูกต้อง การบรรยายการทำงานที่อยู่ภายในส่วน ของการบรรยายสถาปัตยกรรม จะมีการทำงานที่พร้อมเพรียงกันเสมอ หรือแม้แต่โปรเซสซึ่งมีการทำงานภายในเป็น แบบลำดับคำสั่งก็ตาม ซึ่งหากมีหลายๆ โปรเซสอยู่ภายในโครงสร้างเดียวกัน ทุกๆ โปรเซสก็จะทำงานไปพร้อมๆ กัน ด้วย

- สัญญาณและตัวแปร

สัญญาณมีลักษณะเป็นเสมือนตัวกลางฮาร์ดแวร์ที่ใช้ในการส่งผ่านข้อมูลและมีเรื่องของเวลาเข้ามาเกี่ยวข้องด้วยการ กำหนดค่าให้กับสัญญาณจะใช้สัญลักษณ์ <= ในการส่งค่าและสามารถใช้คำสั่ง AFTER เพื่อกำหนดช่วงเวลาในการ ส่งผ่านค่าของสัญญาณ เช่น w <= a AFTER 12 NS หมายถึงการกำหนดค่าสัญญาณ a ให้กับ w หลังจากเวลา ผ่านไป 12 นาโนวินาที ในทางตรงข้ามตัวแปรมีลักษณะเป็นเสมือนตัวกลางที่ใช้ในการส่งผ่านข้อมูลและไม่มีเรื่องของ เวลาเข้ามาเกี่ยวข้องด้วย ซึ่งตัวแปรจะถูกใช้ในส่วนที่มีการทำงานเป็นแบบลำดับคำสั่งเช่นใน ฟังก์ชัน โปรซีเจอร์ และ โปรเซส สำหรับการกำหนดค่าให้กับตัวแปรจะใช้สัญลักษณ์ :=

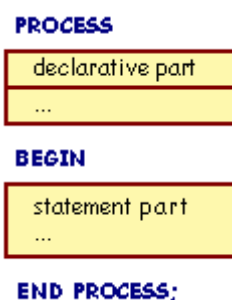
2.7.4 การบรรยายเชิงพฤติกรรม

การบรรยายลักษณะการทำงานของอุปกรณ์ฮาร์ดแวร์ในเชิงพฤติกรรม เป็นการบรรยายลักษณะการเปลี่ยนแปลงของ ข้อมูลในรูปแบบของอัลกอริธึมสำหรับการคำนวณผลลัพธ์ที่เกิดขึ้นซึ่งสืบเนื่องมาจากการเปลี่ยนแปลงสถานะของข้อมูล ที่เข้ามาโดยไม่คำนึงถึงลักษณะ โครงสร้างหรือความสัมพันธ์ของอุปกรณ์ที่อยู่

ภายในว่าจะเป็นอย่างไร ในหัวข้อนี้จะ แสดงถึงการบรรยายเชิงพฤติกรรม แทนการใช้โมดูลฮาร์ดแวร์รวมถึงข้อกำหนดต่างๆ ที่ควรรู้

2.7.5 โพรเซส

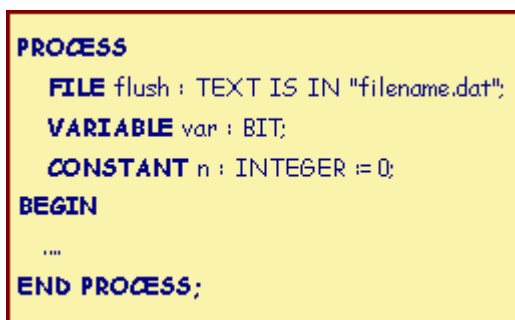
โพรเซสเป็นรูปแบบพื้นฐานอย่างหนึ่งที่ใช้ในการกำหนดให้กับสัญญาณ โพรเซสจะอยู่ในสถานะที่เตรียมพร้อมอยู่เสมอ และจะปฏิบัติคำสั่งพร้อมๆ กันกับโพรเซสอื่นๆ ที่อยู่ในสถาปัตยกรรมบรรยายเดียวกัน โดยโพรเซสจะปฏิบัติงานตามคำสั่งทันทีที่มีเหตุการณ์เกิดขึ้นกับสัญญาณที่อยู่ทางด้านขาเข้าของสัญญาณ กำหนดค่าให้กับสัญญาณ ($\leftarrow$) การบรรยาย โพรเซสจะเริ่มต้นด้วยคำสั่ง PROCESS และจบด้วยคำสั่ง END PROCESS ในรูปที่ 2-25 เป็นการแสดงส่วน ประกอบของการบรรยายแบบโพรเซส ซึ่งประกอบด้วยส่วนของการประกาศตัวแปรที่ต้องใช้และส่วนของการปฏิบัติ คำสั่งเพื่อให้ได้ผลลัพธ์ที่ต้องการ



รูปที่ 2-25 รูปแบบของการบรรยายแบบโพรเซส

2.7.6 การกำหนดตัวดำเนินการภายในโพรเซส

ตัวดำเนินการภายในโพรเซสมี 3 ชนิดคือ ตัวแปร (Variable) ไฟล์ (File) และตัวคงที่ (Constant) ซึ่งตัวดำเนินการทั้งสามชนิดนี้หากมีการประกาศไว้ในโพรเซสใดก็จะใช้ได้เฉพาะภายในโพรเซสนั้นเท่านั้นสำหรับการติดต่อกับภายนอกหรือระหว่างโพรเซสสามารถทำได้โดยใช้สัญญาณ (Signal) หรือตัวคงที่ที่ได้ประกาศไว้ในส่วนของ ARCHITECTURE ในรูปที่ 2-26 แสดงตัวอย่างการประกาศตัวกระทำภายในโพรเซส ซึ่งจะอยู่ระหว่างคำสั่ง PROCESS และ BEGIN และคำสั่งเริ่มต้นที่ถูกกำหนดให้กับตัวดำเนินการภายในโพรเซสจะถูกนำมาใช้ในตอนเริ่มต้น ของการปฏิบัติเพียงครั้งเดียวเท่านั้น ต่างกับคำสั่งเริ่มต้นที่อยู่ภายในโปรแกรมย่อยจะถูกนำมาใช้ทุกครั้งที่มีการเรียกใช้ โปรแกรมย่อยนั้นๆ



รูปที่ 2-26 ตัวอย่างการประกาศตัวดำเนินการภายในโพรเซส

2.7.7 การกำหนดการกระทำภายในโปรเซส

การกระทำใดๆ ภายในโปรเซสจะเป็นการปฏิบัติแบบลำดับ (Sequential) เสมอ ซึ่งภายในโปรเซสสามารถใช้ประโยค เงื่อนไขหรือการซ้ำได้เช่น IF-THEN - ELSE,CASE - WHEN, FOR LOOP และ WHILE LOOP ดังตัวอย่างในรูปที่ 2-27 และ 2-28

```

ARCHITECTURE demo OF partial_process IS
  ...
BEGIN
  PROCESS
    ...
    BEGIN
      ...
      x <= '1';
      IF x = '1' THEN
        perform action_1
      ELSE
        perform action_2
      END IF;
      ...
    END PROCESS;
  END demo;

```

รูปที่ 2-27 เงื่อนไขการกระทำในโปรเซส

```

ARCHITECTURE demo OF partial_process IS
  ...
BEGIN
  PROCESS
    BEGIN
      ...
      x <= a AFTER 10 NS;
      y <= b AFTER 6 NS;
      ...
    END PROCESS;
  END demo;

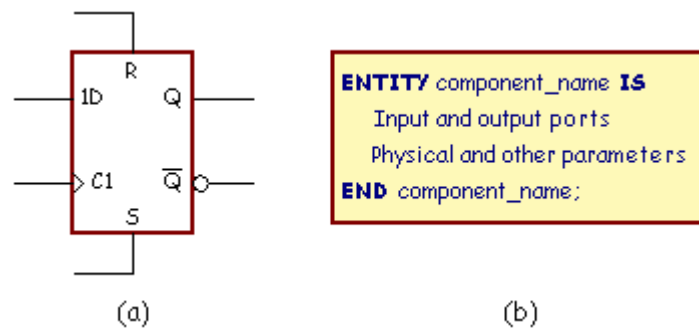
```

รูปที่ 2-28 แสดงการกระทำในโปรเซส

2.7.8 การกระตุ้นและยับยั้งการกระทำของโปรเซส

การกระทำภายในโปรเซสจะอยู่ในสภาพเตรียมพร้อม และมีการปฏิบัติงานอยู่ตลอดเวลาที่มีการเปลี่ยนแปลงของเหตุการณ์ เกิดขึ้น อย่างไรก็ตามเราสามารถกระตุ้นหรือยับยั้งการกระทำภายในโปรเซสได้โดยการกำหนดรายการของสัญญาณที่ต้อง การให้โปรเซสปฏิบัติงานเมื่อมีเหตุการณ์เกิดขึ้นกับสัญญาณที่กำหนดไว้เท่านั้น ส่วนเหตุการณ์ใดๆ ที่เกิดขึ้นกับสัญญาณ ที่ไม่ได้กำหนดไว้ในรายการก็จะไม่ส่งผลให้มีการกระทำ

ภายในโปรเซส ซึ่งรายการของสัญญาณนี้เรียกว่า Sensitivity List และจะกำหนดไว้ภายในวงเล็บหลังคำสั่ง PROCESS รูปที่ 2-29 (a) แสดงตัวอย่างโมเดล และรูปที่ 2-29 (b) เป็นตัวอย่างการบรรยายการเชื่อมต่อของ D-Flip Flop ส่วนรูปที่ 2-30 แสดงถึงการบรรยายเชิงพฤติกรรมของ D-Flip Flop โดยในรูปที่ 2-30 (a) เป็นการใช้อั้วกระทำภายนอกโปรเซส และรูปที่ 2-30 (b) เป็นการใช้อั้วกระทำภายในโปรเซส โดยมีรายการของสัญญาณ (rst, set, clk) เป็นตัวกระตุ้นการปฏิบัติงานภายในโปรเซส



รูปที่ 2-29 (a) ตัวอย่างโมเดล D-Flip Flop (b) การบรรยายการเชื่อมต่อของ D-Flip Flop

```

ARCHITECTURE behavioral OF d_sr_flipflop IS
    SIGNAL state : BIT := '0';
BEGIN
    dff : PROCESS (rst, set, clk)
        BEGIN
            IF set = '1' THEN
                state <= '1' AFTER sq_delay;
            ELSIF rst = '1' THEN
                state <= '0' AFTER rq_delay;
            ELSIF clk = '1' AND clk ' EVENT THEN
                state <= d AFTER cq_delay;
            END IF;
        END PROCESS dff;
    q <= state;
    qb <= NOT state;
END behavioral;
  
```

(a)

```

ARCHITECTURE average_delay_behavioral OF d_sr_flipflop IS
BEGIN
    dff : PROCESS (rst, set, clk)
        VARIABLE state : BIT := '0';
        BEGIN
            IF set= '1' THEN
                state <= '1';
            ELSIF rst = '1' THEN
                state <= '0';
            ELSIF clk = '1' AND clk ' EVENT THEN
                state <= d;
            END IF;
            q <= state AFTER (sq_delay + rq_delay + cq_delay)/3;
            qb <= NOT state AFTER(sq_delay + rq_delay + cq_delay)/3;
        END PROCESS dff;
    END behavioral;

```

(b)

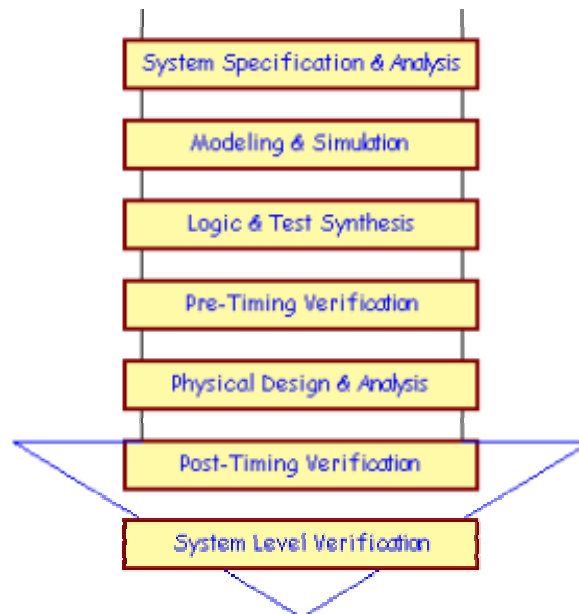
รูปที่ 2-30 การบรรยายเชิงพฤติกรรมของ D-FlipFlop

(a) การใช้ตัวกระทำภายนอกโปรเซส

(b) การใช้ตัวกระทำภายในโปรเซส

2.7.9 การออกแบบจากบนลงล่าง

ในการพัฒนางจรรวมดิจิทัลขนาดใหญ่ที่มีความซับซ้อน วิศวกรหรือผู้ออกแบบมักจะมองการออกแบบให้อยู่ในรูปของ บล็อกไดอะแกรมก่อนที่จะทำวิเคราะห์ให้ลึกถึงรายละเอียดต่อไป ซึ่งภาษา VHDL นั้นอนุญาตให้อธิบายและวิเคราะห์การทำงานของแต่ละบล็อก รวมถึงการปรับปรุงการทำงานจากผลที่วิเคราะห์เพื่อให้ได้การทำงานตามต้องการ นอกจากนี้ยังสามารถเพิ่มเติมในรายละเอียดในแต่ละขั้นตอนได้ ซึ่งหลักการนี้สอดคล้องกับหลักการออกแบบจากบนลงล่าง (Top - Down Design) นั่นเอง ถ้าทดลองเปรียบเทียบกับการออกแบบจากล่างขึ้นบน (Bottom - Up Design) จะเห็นได้ว่า การออกแบบจากล่างขึ้นบนจะใช้เวลาการออกแบบมากกว่า 90% เนื่องจากการวาดวงจรด้วยอุปกรณ์ต่างๆ (Schematic capture) ที่ประกอบกันเข้าเป็นวงจรที่ต้องการออกแบบ ก่อนแล้วจึงทำการจำลองการทำงาน และตรวจ สอบความถูกต้อง ดังนั้นการใช้ภาษา VHDL กับหลักการออกแบบจากบนลงล่างจึงเป็นทางออกให้กับวิศวกรให้สามารถ ออกแบบและพัฒนาวงจรที่มีความซับซ้อนได้มากขึ้น ทั้งยังช่วยลดเวลาและค่าใช้จ่ายในการออกแบบด้วย



รูปที่ 2-31 ขั้นตอนการออกแบบจากบนลงล่าง

จากรูปที่ 2-31 แสดงถึงขั้นตอนของการออกแบบจากบนลงล่าง ทั้งนี้ในทางปฏิบัติอาจมีข้อแตกต่างไปจากนี้บ้าง เล็กน้อยเนื่องจากขั้นตอนของการผลิต (Implementation) สามารถกระทำได้หลายเทคโนโลยี สำหรับรายละเอียดของขั้นตอน การออกแบบจากบนลงล่างในแต่ละขั้นตอนมีดังนี้

2.7.9.1 สร้างข้อกำหนดของความต้องการ และวิเคราะห์ระบบ เพื่อหาแนวความคิดและหลักการ (Idea and Concept) ใน การแก้ปัญหา

2.7.9.2 เขียนรูปแบบของระบบที่ต้องการออกแบบโดยใช้ภาษา VHDL หรือ ภาษา HDL อื่น ๆ สำหรับบรรยายพฤติกรรมการทำงาน พร้อมทั้งจำลองการทำงาน เพื่อเปรียบเทียบและตรวจสอบความถูกต้องกับข้อกำหนด

2.7.9.3 หลังจากที่ได้หลักการขั้นต้นพร้อมแนวความคิดที่ผ่านการตรวจสอบแล้วหลักการนี้จะถูกเพิ่มเติมในรายละเอียดลงมาเป็นลำดับขั้นที่สอง จนกระทั่งอยู่ในระดับที่จะนำไปผลิตจริง หรือสังเคราะห์ในขั้นตอนนี้อะเทคโนโลยีที่จะมารองรับ วงจรรออกแบบจะถูกกำหนดขึ้น และระบบช่วยการออกแบบจะสังเคราะห์วงจรที่ได้จากรูปแบบที่เขียนขึ้นให้อยู่ในรูปของ วงจรที่ประกอบด้วยอุปกรณ์อิเล็กทรอนิกส์ หรือ วงจรในระดับเกต และการเชื่อมต่อระหว่างกันของอุปกรณ์เหล่านั้นหรือ ไม่ก็อยู่ในรูปของ Netlist ที่สามารถนำไปผลิตในอุปกรณ์อื่นได้

2.7.9.4 หลังจากการสังเคราะห์วงจรให้อยู่ในระดับเกตหรือ Netlist แล้ว ข้อมูลนี้จะถูกใช้สำหรับจำลองการทำงานในเรื่อง ความถูกต้องของฟังก์ชัน พร้อมกับนำข้อมูลที่เกี่ยวข้องกับเวลาเข้ามาประกอบการพิจารณาด้วย ซึ่งตามปกติแล้วอุปกรณ์ ทางอิเล็กทรอนิกส์ทุกชิ้นจะมีเวลาหน่วงของการแพร่กระจาย (Propagation Delay Time) เสมอ ถึงแม้ว่าจะเป็น เวลาที่น้อยมากในระดับนาโนวินาทีก็ตาม แต่ถ้าภายในวงจรหนึ่งประกอบด้วยเกตของฟังก์ชันต่างๆ จำนวน 10,000 เกต ขึ้นไป เวลาดังกล่าวนี้จะสะสมกันมากขึ้น จนอาจทำให้การทำงานของวงจรรวมทั้งหมดผิดพลาดไป หรือไม่สามารถทำงานในย่านความถี่สัญญาณนาฬิกาที่สูงได้

2.7.9.5 ผลิตเป็นวงจรจริง (Technology and device mapping) โดยนำข้อมูลที่ได้จากการสังเคราะห์มาผลิต ซึ่งอาจ จะอยู่ในรูปของแผงวงจรไฟฟ้า ที่ประกอบด้วยอุปกรณ์หลายๆ ชิ้นหรืออยู่ในรูปของวงจรรวม ASIC

2.7.9.6 ทำการตรวจสอบการทำงานและตัวแปรทางด้านเวลาทั้งหมด เพื่อความถูกต้องของวงจรเป็นครั้งสุดท้ายก่อนนำไปรวมเข้ากับอุปกรณ์อื่นๆ ให้เป็นระบบดิจิทัล เนื่องจากในขั้นตอนนี้ วงจรที่ออกแบบ จะประกอบด้วยจุดต่อทางอินพุตและเอาต์พุต ซึ่งเป็นจุดต่อสำหรับการรับและส่งสัญญาณกับภายนอก

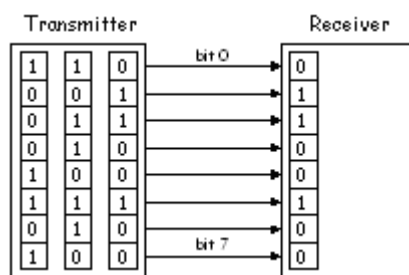
2.7.9.7 นำวงจรที่ออกแบบไว้ประกอบเข้ากับอุปกรณ์อื่นๆ ให้เป็นระบบที่สมบูรณ์ แล้วทำการทดสอบการทำงานทั้งระบบร่วมกับอุปกรณ์อื่นๆ อีกครั้งเพื่อควบคุมคุณภาพของผลิตภัณฑ์

2.8 การรับ – ส่งข้อมูลแบบอนุกรมด้วย FPGA

ในการสื่อสารหรือการส่งข้อมูลในระบบดิจิทัลนั้นมีรูปแบบในการสื่อสารที่สำคัญอยู่ 2 แบบ คือ

- การสื่อสารแบบขนาน (Parallel Communication)

การสื่อสารในรูปแบบที่ส่งข้อมูลออกไปทีละหลายๆ กันทั้ง 8 bits ผ่านสายสัญญาณทั้ง 8 เส้น การสื่อสารและการส่งข้อมูลแบบขนานจะแสดงให้เห็นดังรูปที่ 2-32

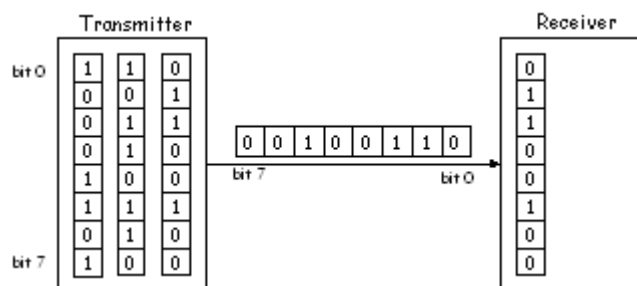


รูปที่ 2-32 ลักษณะการสื่อสารแบบขนาน

จะเห็นได้ว่าการสื่อสารแบบขนานมีข้อดีคือทำให้สามารถส่งข้อมูลที่ละมากๆ และรวดเร็วกว่าการส่งแบบอนุกรม แต่การสื่อสารแบบขนานมีข้อจำกัดคือ ไม่สามารถส่งข้อมูลในระยะไกลๆ ได้และยังต้องใช้สายสัญญาณหลายเส้นในการส่งข้อมูล ทำให้สิ้นเปลืองกว่าการสื่อสารแบบอนุกรม ทำให้ไม่สะดวกในการใช้งาน ตัวอย่างของการสื่อสารแบบขนาน เช่น เครื่องพิมพ์ (Printer) และการสื่อสารทางพอร์ตนาน (ECP Printer Port) เป็นต้น

- การสื่อสารแบบอนุกรม (Serial Communication)

เป็นการสื่อสารโดยการส่งข้อมูลที่ละบิต ผ่านสายสัญญาณเส้นเดียว จนครบทั้ง 8 bits หรือ ไบท์ โดยจะส่งบิตต่ำ (LSB) ออกไปก่อน สามารถแสดงให้เห็นหลักการส่งข้อมูล แบบอนุกรมได้ดังรูปที่ 2-33



รูปที่ 2-33 การส่งข้อมูลแบบอนุกรม

2.8.1 การอินเทอร์เฟสตามมาตรฐาน RS-232

มาตรฐาน RS-232 เป็นมาตรฐานที่ได้รับการพัฒนามานานและถูกใช้งานกันอย่างแพร่หลาย เราใช้ RS-232 เชื่อมต่อ DTE (Data Terminal Equipment) เข้ากับ DCE (Data Communication Equipment) เช่นการต่อเทอร์มินัลเข้ากับโมเด็ม มาตรฐาน RS-232 กล่าวถึงลักษณะทางกล, ลักษณะของสัญญาณไฟฟ้าและลักษณะการทำงานที่ใช้การอินเทอร์เฟส ตัวอย่างของอุปกรณ์ที่ใช้ในการอินเทอร์เฟสตามมาตรฐาน RS-232 ได้แก่ เทอร์มินัล , พล็อตเตอร์ , ลอจิกแอนาไลเซอร์ (Logic Analyzer) และเครื่องพิมพ์ ถ้าการประยุกต์ใช้งานของเราต้องการทำอินเทอร์เฟสอุปกรณ์ เข้ากับอินเทอร์เฟสมาตรฐาน RS-232 เราจำเป็นต้องแปลงระดับสัญญาณ TTL ให้เป็นระดับสัญญาณแบบอื่น ซึ่งรายละเอียดของระดับสัญญาณที่ใช้สำหรับ RS-232 จะได้กล่าวต่อไป

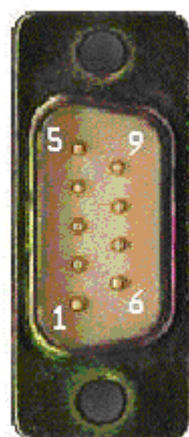
2.8.2 ลักษณะสัญญาณที่ใช้ในการอินเทอร์เฟส

มาตรฐาน RS-232 ใช้สัญญาณเพียงเส้นเดียวในการส่งสัญญาณ โดยสัญญาณจะส่งไปในทิศทางเดียวกัน ในกรณีที่อัตราเร็วในการส่งข้อมูลมีค่าเท่ากับ 20 kbps (กิโลบิตต่อวินาที) ซึ่งค่านี้เป็นค่าสูงสุดที่ใช้ในการสื่อสารข้อมูล (ในปัจจุบันพัฒนาให้สามารถส่งข้อมูลได้มากกว่านี้) ระยะทางในการส่งข้อมูลไม่ควรเกิน 50 ฟุต (ตามข้อกำหนดในมาตรฐาน) สำหรับการแทนแรงดันของระดับสัญญาณจะแทนระดับสัญญาณของลอจิก “0” ด้วยค่าแรงดัน +3 โวลต์ ถึง +12 โวลต์ ส่วนลอจิก “1” จะแทนระดับสัญญาณด้วยค่าแรงดันระหว่าง -3 โวลต์ ถึง -12 โวลต์

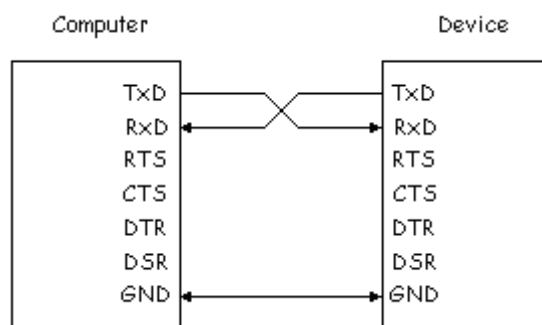
2.8.3 การออกแบบตัวแปลงสัญญาณ

การเชื่อมต่อกับพอร์ตสื่อสารของคอมพิวเตอร์ส่วนบุคคลจะเลือกใช้พอร์ตสื่อสารแบบ อนุกรม 9 ขา (DB-9) ซึ่งสามารถทำการรับส่งข้อมูลได้แบบอนุกรม โดยลักษณะของสัญญาณจะเป็นไปตามมาตรฐาน RS-232 โดยลักษณะของการเชื่อมต่อของพอร์ตสื่อสารสำหรับคอนเน็คเตอร์แบบ DB-9 สามารถแสดงให้เห็นได้ดังรูปที่ 2-34 และ รูปที่ 2-35

ตำแหน่งขา DB-9	สัญญาณ
1	Data Carrier Detect : DCD
2	Received Data : RxD
3	Transmitted Data : TxD
4	Data Terminal Ready : DTR
5	Signal Ground : GND
6	Data Set Ready : DSR
7	Request To Send : RTS
8	Clear To Send : CTS
9	Ring Indicator : RI



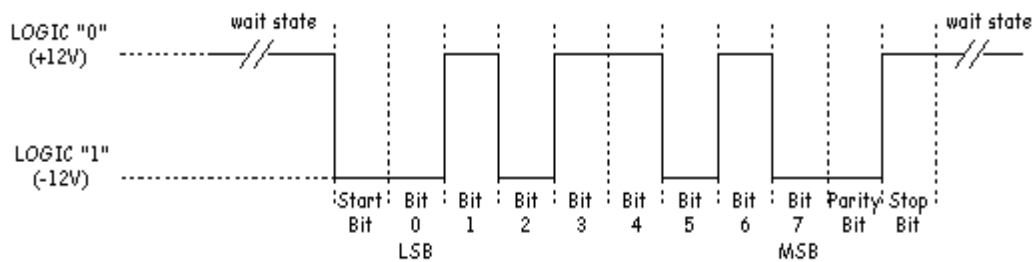
รูปที่ 2-34 การจัดขาของคอนเน็คเตอร์พอร์ตอนุกรมแบบ DB-9



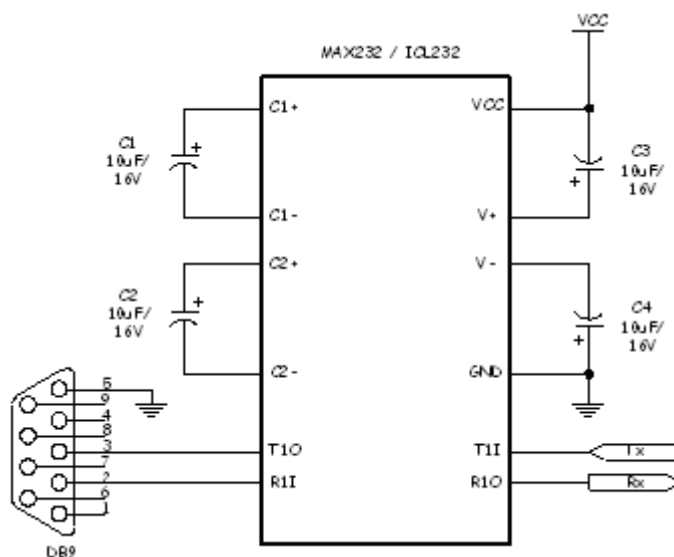
รูปที่ 2.35 การต่ออุปกรณ์ภายนอกกับคอมพิวเตอร์โดยใช้สัญญาณเพียง 3 เส้น

2.8.4 วงจรอแดปเตอร์

จากหัวข้อที่ผ่านมาตอนนี้เราทราบแล้วว่าในการสื่อสารแบบอนุกรมนั้น ในระดับของลอจิก “0” จะแทนระดับลอจิกด้วยระดับแรงดันระหว่าง +3 โวลต์ ถึง +12 โวลต์และในระดับของลอจิก “1” จะแทนระดับลอจิกด้วยระดับแรงดันระหว่าง -3 โวลต์ ถึง -12 โวลต์ เพราะฉะนั้นต้องทำการแปลง ระดับของลอจิก “1” และ “0” ให้เป็นระดับแรงดันดังกล่าว ซึ่งจะต้องใช้วงจรในการแปลงระดับแรงดัน สามารถแสดงลักษณะการส่งข้อมูลแบบอนุกรมที่ผ่านตัวแปลงแรงดันได้ดังรูปที่ 5 ในส่วนของวงจรแปลงระดับแรงดันสำหรับการสื่อสาร RS-232 นั้นจะใช้ไอซีเบอร์ MAX232 หรือ ICL232 ดังรูปที่ 2-37



รูปที่ 2-36 รูปแบบการสื่อสารแบบอนุกรม



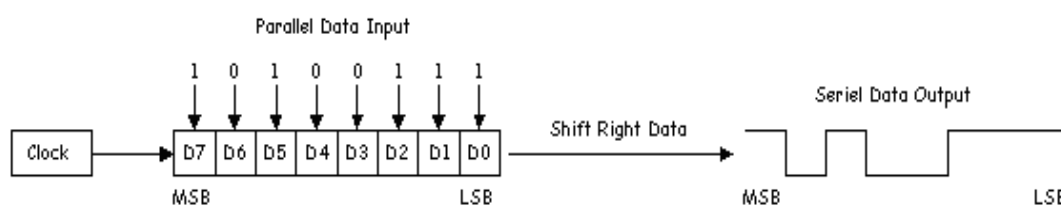
รูปที่ 2-37 วงจรแปลงระดับแรงดันของการสื่อสารแบบอนุกรม

2.8.5 รูปแบบของการส่งข้อมูลแบบอนุกรมและอัตราบอดในการสื่อสาร

อัตราบอด (Baud Rate) คือความเร็วในการรับ - ส่งข้อมูลอนุกรมมีหน่วยเป็นบิตต่อวินาที ซึ่งจะบอกจำนวนบิตที่รับ - ส่งในเวลา 1 วินาที เช่น ส่งข้อมูลด้วยอัตรา 9600 บิตต่อวินาที ก็คือการส่งข้อมูลตัวอักขระขนาด 10 บิต (บิต Start 1 บิต บิตข้อมูล 8 บิต บิต Stop 1 บิต) ได้ 960 ตัวอักขระใน 1 วินาที ซึ่งตารางที่ 1 แสดงอัตราบอดของ UART ที่ใช้กันทั่ว ตารางที่ 1 เป็นการแสดงอัตราบอดทั่วไปที่ใช้ในการโอนย้ายข้อมูลแบบอนุกรม

อัตราบอด	ช่วงเวลาของแต่ละบิต
110	9.91 ms
150	6.67 ms
300	3.33 ms
600	1.67 ms
1200	0.833 ms
2400	0.417 ms
4800	0.208 ms
9600	0.104 ms
19200	0.052 ms

ตารางที่ 2-2 อัตราบอดที่ใช้กันทั่วไป



รูปที่ 2-38 การแปลงข้อมูลแบบขนานเป็นข้อมูลแบบอนุกรม

ในรูปที่ 2-38 เป็นการแปลงข้อมูลแบบขนานเป็นข้อมูลแบบอนุกรม โดยเริ่มแรกข้อมูลแบบขนานจะถูกนำไปเก็บไว้ใน Shift Register หลังจากนั้นจะใช้สัญญาณนาฬิกาในการเลื่อนค่าในรีจิสเตอร์ออกมาทีละบิต (โดยการเลื่อนค่าไปทางขวามือ) โดยบิตแรกที่ถูกเลื่อนออกมาคือบิต LSB ของข้อมูลและบิตที่สองที่ถูกเลื่อนออกมาคือบิตที่อยู่ถัดจากบิต LSB และบิตต่อไป สำหรับบิตสุดท้ายที่ถูกเลื่อนออกมาคือบิต MSB ของข้อมูล

การแปลงข้อมูลแบบอนุกรมไปเป็นข้อมูลแบบขนานนั้นจะมีขั้นตอนตรงกันข้ามกับที่กล่าวมา นั่นคือข้อมูลแบบอนุกรมจะถูกเคลื่อนเข้าไปใน Shift Register โดยใช้สัญญาณนาฬิกาเป็นตัวควบคุม และหลังจากที่มีการเคลื่อนข้อมูลทุกบิตเข้าไปใน Shift Register ได้หมดแล้ว ข้อมูลในรีจิสเตอร์นี้ จะถูกนำออกมาแบบขนานเพื่อนำไปใช้งานต่อไป

อุปกรณ์ที่ทำหน้าที่แปลงข้อมูลแบบอนุกรมเป็นข้อมูลแบบขนานและแปลงข้อมูลแบบขนานเป็นข้อมูลแบบอนุกรมเรียกว่า UART (Universal Asynchronous Receiver-Transmitter) ซึ่งนอกจากจะมีหน้าที่ในการแปลงข้อมูลแล้ว UART ยังมีหน่วยควบคุมและหน่วยตรวจสอบการทำงานอีกด้วย ในการส่งข้อมูลขนาด 8 บิต แบบอนุกรมนี้จะต้องมีบิตเริ่มต้น (Start Bit) และบิตหยุด (Stop Bit) เพิ่มขึ้นมาซึ่งจะทำให้ข้อมูลที่ถูกส่งไปจริงๆ นั้นมีขนาด 10 บิต